

UNIVERSIDAD NACIONAL

Facultad de Ciencias Exactas y Naturales

ESCUELA DE INFORMÁTICA

**“Propuesta de un algoritmo de visión por computadora para la caracterización de
abejas *Apidae Meliponini* a través de dispositivos móviles”**

Para optar al grado de Licenciado en Informática
con énfasis en Sistemas WEB

Ing. José Pablo Araya Pérez

Heredia, Costa Rica

TABLA DE CONTENIDOS

CAPÍTULO I: INTRODUCCIÓN.....	7
1. Antecedentes.....	7
2. Planteamiento del problema.....	10
3. Justificación	12
4. Objetivos del Proyecto.....	16
4.1 <i>Objetivo general</i>	16
4.2 <i>Objetivos específicos</i>	16
CAPÍTULO II: MARCO TEÓRICO.....	18

CAPÍTULO III: METODOLOGÍA.....	32
1. Tipo de investigación.....	32
2. Población y muestra.....	33
3. Descripción de instrumentos.....	34
4. Procedimientos para analizar la información del diagnóstico	34
CAPÍTULO IV: PROPUESTA DE SOLUCIÓN.....	37
1. Propuesta de solución	37
2. Validación de la propuesta.....	52
CAPÍTULO V: CONCLUSIONES Y RECOMENDACIONES	84
Conclusiones	84
Limitaciones.....	85
Recomendaciones	86
REFERENCIAS	89
Anexo 1 – Manual de Usuario	96
Notas para el administrador y/o desarrollador	97
Inicio	99
Login.....	101
Logout.....	102
Ayuda.....	103
Menú Principal.....	104
<i>Consultar Galería</i>	<i>105</i>
<i>Herramienta de filtrado de búsqueda</i>	<i>106</i>
<i>Capturar y Analizar</i>	<i>108</i>
Menú de Abejas	113
<i>Agregar Abeja</i>	<i>114</i>
<i>Consultar Abeja</i>	<i>115</i>
<i>Actualizar Abeja.....</i>	<i>115</i>
<i>Eliminar Abeja</i>	<i>118</i>
<i>Generar Imágenes.....</i>	<i>119</i>
Menú de Usuarios	119
<i>Tipos de Usuarios</i>	<i>120</i>
<i>Agregar Usuarios.....</i>	<i>121</i>
<i>Consultar Usuarios</i>	<i>122</i>
<i>Actualizar Usuario.....</i>	<i>123</i>
<i>Eliminar Usuarios.....</i>	<i>123</i>
Referencias útiles	124

Índice de Tablas

Tabla 1. Clasificación Meliponini	18
Tabla 2. Descripción de peticiones.....	55
Tabla 3. Tipo de usuarios y privilegios	60
Tabla 4 . Resumen de pruebas template matching	76
Tabla 5 . Resumen de metodos template matching	77
Tabla 6 . Resumen resultados Cascade Classifier	78
Tabla 7. Matriz de confusión Cascade Classifier	81

Índice de Figuras

Figura 1 . Clasificación Meliponini.....	19
Figura 2. Flujo Propuesta	35
Figura 3 Ramificación de ejemplo git	41
Figura 4 . Modelo Cliente-Servidor	42
Figura 5 . Ejemplificación Template Matching.....	44
Figura 6 . Ejemplificación Cascade Classifier	46
Figura 7 . Ejemplo de centralización de imágenes negativas.....	47
Figura 8. Ejemplo de centralización de imágenes positivas.....	48
Figura 9 . Ejemplo feature buscado con Cascade Classifier	49
Figura 10 . Ejemplo de identificación de abejas con Cascade Classifier	49
Figura 11 . Servidor basado en Django en ejecución	53
Figura 12 . Índice del API creado.....	54
Figura 13 . Login vista web.....	56
Figura 14 . Login vista móvil	57
Figura 15 . Menú principal vista web.....	58
Figura 16 . Menú de usuarios vista web.....	58
Figura 17 . Menú de abejas vista web	59
Figura 18 . Generación de imágenes por folder	60
Figura 19 . Menú principal vista móvil regular.....	61
Figura 20 . Captura vista móvil.....	62

Figura 21 . Mensaje analizando imagen.....	63
Figura 22 . Resultado de análisis vista móvil.....	64
Figura 23 . Ejemplo ficha técnica.....	65
Figura 24 . Consultar abeja vista móvil.....	66
Figura 25 . Función para capturar imagen.....	67
Figura 26 . Función para convertir a png	68
Figura 27 . Función crear imagen temporal y conversiones.....	68
Figura 28 . Función de clasificación y lista de aserción.....	69
Figura 29 . Procesamiento de imágenes	69
Figura 30 . Identificación y cuadriculación de puntos de interés.....	70
Figura 31 . Diagrama detección de objetos.	70
Figura 32 . Imagen base ejemplo 1.....	71
Figura 33 . Imagen de entrada ejemplo 1	72
Figura 34 . Análisis y captura template matching 1	73
Figura 35 . Resultado Template Matching ejemplo 1	74
Figura 36 . Base 1 trigona silvestriana	75
Figura 37 . Entrada 2 trigona silvestriana.....	75
Figura 38 . Resultados trigona silvestriana 2	75
Figura 39 . Entrada trigona silvestriana 3.....	76
Figura 40 Resultado Trigona Silvestriana 3	76
Figura 41 . Ejemplo Ruido 1	85
Figura 42 . Ejemplo Ruido 2	85

CAPÍTULO I

INTRODUCCIÓN

CAPÍTULO I: INTRODUCCIÓN

1. Antecedentes

En Costa Rica las empresas productoras de semillas cultivan plantas ornamentales en áreas amplias rodeadas de redes a pruebas de insectos, estos productores tienen una alta demanda de polinizadores, ya que una ventaja de trabajar con ellos es que se pueden procesar incluso plantas fuera de temporada. Las abejas por su naturaleza son agentes polinizadores, sin embargo, las más comunes son africanizadas y por defecto son más defensivas, por lo cual son muy poco utilizadas para la polinización en invernaderos (*Slaa, Sánchez, Braga, Hofstede, 2006*).

Las abejas cumplen un papel importante en la producción de la miel, debido a que este producto es considerado hoy en día como “el oro líquido”. Sus beneficios son muy grandes y diversos, desde la gastronomía, farmacéutica e incluso industria cosmética (*Redacción Capital México, 2017*). Algunos beneficios de la miel son:

- La miel nunca caduca, es un gran conservante natural en donde las bacterias y levaduras no pueden sobrevivir, debido a la alta concentración de azúcar y humedad que contiene.
- Debido a sus propiedades antisépticas y antimicrobianas prevén infecciones y funciona como cicatrizante.
- Por su alta concentración de azúcar, funciona como energizante. Cabe mencionar que son azúcares simples, por lo que su absorción es acelerada.
- Es curativo, al beneficiar y suavizar la garganta cuando hay tos o irritación.

- Funciona como base para la creación de champú y máscaras para pestañas. (*Botanical Online, 1999-2018*).

Sin embargo, no todas las especies de abejas son aptas para realizar trabajos de polinización, conservación de plantas y producción de miel, ante esta situación los apicultores utilizan ambientes controlados donde normalmente se encuentran las abejas sin aguijón, cuyo nombre científico es *Apidae Meliponini*, por tener las mejores características para este tipo de labor.

La categorización taxonómica de las abejas se ha convertido en un verdadero objeto de estudio, ya que a nivel nacional se cuentan con alrededor de 60 especies, lo que lo convierte en un 15% de la diversidad mundial (*Chavarría, 2015*). Esto conlleva a su preservación e investigación, para poder hacer una explotación racional.

Al igual que otras especies de animales, las abejas se dividen en diferentes estratos. Todas las especies de abejas pertenecen al reino animalia, filo *arthropoda*, clase *insecta* y orden *hymenoptera* (*Chavarría, 2015*). Hasta aquí, las abejas comparten todos los estratos anteriores con otros insectos tales como los abejorros, avispa y hormigas. A partir de la familia es donde se empieza a categorizar solamente abejas. Luego de las familias se encuentran las subfamilias y tribus tales como las *Apini* y *Meliponini*. Cada tribu consta de diferentes géneros para así llegar finalmente hasta las diferentes especies.

Las especies de abejas sin aguijón o *Apidae Meliponini*, es uno de los grupos taxonómicos mejor estudiados y los cuales pueden ser considerados como buenos polinizadores en cultivos de origen neo tropical (*Martínez, Merlo, 2014*), lo cual justifica el interés del Centro de Investigaciones Apícolas Tropicales (CINAT) por el estudio de las abejas pertenecientes a esta tribu.

El Centro de Investigaciones Apícolas Tropicales (CINAT-UNA), es una institución de la Facultad de Ciencias de la Tierra y el Mar (FCTM) de la Universidad Nacional de Costa Rica. Es un centro especializado en el estudio de las abejas tropicales, que, mediante la investigación, docencia, extensión y prestación de servicios, desde una perspectiva interdisciplinaria, promueve el desarrollo de una apicultura y meliponicultura sostenible en Costa Rica y Centroamérica.

Hay varios beneficios de tener conocimiento de las abejas sin aguijón, partiendo del hecho de que gran cantidad de esta especie puede ser administrada en colmenas artificiales. Otras características es que es un grupo muy adecuado para servicios de polinización debido que las colonias no mueren después de reproducirse, lo cual ayuda a mantener las colmenas por largos periodos de tiempo; luego carecen de un aguijón funcional que las hace especialmente adecuadas para la polinización de cultivos en áreas pobladas y recintos tales como los invernaderos (*Slaa, Sánchez, Braga, Hofstede, 2006*).

A raíz de la problemática presentada por el CINAT de la Universidad Nacional referente a la falta de apoyo tecnológico para la identificación de abejas sin aguijón nativas de Costa Rica, se ha identificado la necesidad de una herramienta que permita la fácil y práctica identificación de abejas *Apidae Meliponini*, estas por los múltiples beneficios que ofrecen y además por la importancia de la preservación y estudio de las especies con las que se cuentan a nivel nacional. Esta carencia conlleva a la investigación en el área apícola, para poder comprender la forma en la que los expertos en el área, logran realizar la especiación actual. Por otro lado, se pretende investigar acerca de las especies *Apidae Meliponini*, de esta forma lograr el conocimiento necesario para analizar la mejor forma de dar solución al problema presentado por el CINAT y plantear objetivos que solventen dicha necesidad.

Dentro de la solución y como parte de los objetivos de este proyecto, se pretende la creación de una aplicación móvil que logre identificar las especies de abejas mencionadas por medio de visión por computadora. De esta forma, brindar una herramienta útil y de fácil

uso para los interesados en el área apícola e incluso meliponicultura del CINAT. La misma podrá ser utilizada por otros usuarios dentro de lo que se puede mencionar: productores, investigadores, instituciones y aficionados al tema.

Cabe mencionar que la movilidad de una aplicación daría la facilidad de iniciar un estudio para la identificación de abejas sin aguijón en cualquier momento y lugar en que se encuentre el usuario. A raíz de la información obtenida, se utilizará como insumo para indagar por medio de la aplicación móvil el tipo de especie ante la cual se encuentra. Además, no solo serviría en un campo de trabajo, sino también en un campo de estudio con el material suficiente para discriminar e investigar la especie que desee dentro de la tribu *Meliponini*.

Una de las ventajas que ofrecería esta herramienta es la portabilidad dentro del campo de trabajo, facilitando la identificación de las abejas en tiempo real. También otro de sus beneficios es que permitiría descartar de una manera más fácil cuál especie es la que se ha identificado, eliminando filtros previos o con un procedimiento de búsqueda en llaves taxonómicas para realizar la búsqueda o rango de posibles especies.

2. Planteamiento del problema

De acuerdo con el CINAT, actualmente los expertos en el área de estudio de las abejas han disminuido, debido a que todo el conocimiento y estudio taxonómico hecho por expertos de alto reconocimiento ha quedado paulatinamente en papel y no se ha hallado una forma de poder brindar dicho conocimiento de una manera fácil y práctica al público. Aunado a ello, existe la falta de incentivos a investigadores para el estudio de las abejas nativas de Costa Rica, específicamente las que pertenecen a la tribu *Meliponini* (Chavarría, 2015). Por lo cual se requiere de un medio para que expertos apícolas, meliponicultores,

estudiantes o incluso aficionados, conozcan y tengan una herramienta de fácil identificación de dichas especies. (Monge I, 2020).

Siguiendo la misma problemática presentado por el CINAT, hoy en día no hay un medio o una plataforma que brinde información, categorice fácilmente a una abeja sin aguijón dentro de su especie, para que cualquier tipo de usuario brinde detalles, presente información completa y referente a cada especie, tanto a nivel de características como de imágenes, ya que puede ser desde un experto, aficionado o hasta un estudiante del área el que indague en un estudio con la información que se le pueda brindar por medio de una plataforma tecnológica. (Monge I, 2020).

Adicionalmente, cabe destacar que Costa Rica cuenta con el 15% de la diversidad mundial de abejas *Apidae Meliponini*, con 59 especies y 20 géneros identificados (Chavarría, 2015), este dato es importante de destacar, ya que con lo anteriormente mencionado y recientemente con base en entrevistas realizadas con el CINAT, hay una clara evidencia de la falta de apoyo tecnológico que tiene el instituto para poder llevar un registro de la información recopilada por años de experiencia y diferentes estudios realizados, de esta manera poder preservarla y utilizarla con mayores fines prácticos. (Gallardo M, 2021).

Dentro de los casos de estudio que comenta el CINAT, se expone un caso muy interesante, como el del árbol de Aguacatillo y la atracción de quetzales que este árbol genera, ya que la preservación de este árbol se origina por el trabajo de polinización que realizan las abejas. Consecuentemente el fruto de este árbol, que dicho sea de paso su nombre se debe al fruto que producen, el cual es un aguacate pequeño, de ahí “aguacatillo”, atrae a diferentes especies de aves y en especial a los quetzales que consumen dicho fruto. Esto es todo un ciclo natural y caso de estudio de preservación originado de nuevo por el trabajo de polinización que inicialmente realizan las abejas. (Gallardo M, 2021).

Debido a esta carencia, se conlleva a realizar una investigación en el área apícola, para poder comprender la forma en la que los expertos en al área logran realizar la especiación actual. Dentro de este proceso, los expertos del instituto comienzan con la utilización de llaves de descarte, para inicialmente ubicar a la abeja, luego con base en diferentes insumos como lo son su piquera, comportamiento, colores, vellosidad, ubicación, características de la miel, características del polen, tamaño, entre otras, lograr a través de las llaves ya previamente establecidas con base a la experiencia e investigaciones realizadas, ir discriminando y descartando antes cuál especie o dentro de cuáles géneros puede ubicarse la abeja de estudio (Gallardo M, 2021). Todo esto proceso es actualmente realizado manualmente y cabe mencionar que muchas veces los resultados y análisis realizados por los expertos puede incluso diferir, ya que las observaciones que realiza cada taxónomo pueden ser diferentes y hasta contradecirse, con esto se denota que es un trabajo arduo y de muchas horas para poder identificar ciertos tipos de especies. Lo anterior debido a que, comparten características minúsculas que no son fácilmente identificables al ojo humano, por lo que los expertos taxónomos, recurren al uso de diferentes instrumentos como el estéreomicroscopio, de esta manera poder visualizar en mayor detalle dichas características que no son fácilmente observables (*Gallardo M. 2021*).

Con base en todos los insumos recopilados durante las entrevistas con los expertos del CINAT, se procede a justificar la necesidad de un apoyo en el área tecnológica, la cual pueda de alguna manera facilitar el trabajo manual que el instituto realiza y posteriormente proponer una solución desde del área informática.

3. Justificación

Las abejas son mundialmente reconocidas como las principales polinizadoras. La acción polinizadora de las abejas está probada como crucial para el ciclo vegetal de la flora, más del 70% de los 100 cultivos que proporcionan la mayoría del alimento para humanos son polinizados por abejas (Álvarez, 2018).

Por características especiales de las especies, el CINAT desea enfocar el desarrollo de una aplicación solamente en las abejas sin aguijón, considerando la cantidad de especies que se pueden encontrar en el país. Según Chavarría (2015) en Costa Rica se pueden encontrar alrededor de 60 especies además de que se encuentran esparcidas por todo el territorio nacional. Debido a una baja en el sistema inmunológico de las abejas con aguijón, las colmenas mueren fácilmente por enfermedades, por lo cual poco a poco resurge la meliponicultura y otras alternativas (Meliponicultura, 2015).

El uso de esta aplicación serviría como primera fuente de información para las personas dedicadas al comercio de cultivos o viveros, ya que datos tales como las plantas de las que se alimentan y a su vez polinizan, varían dependiendo de la especie y la región en la que se encuentren. Algunas especies de *Trigonas* (especie de abeja sin aguijón) se utilizan como polinizadoras de cultivos específicos manejadas a campo abierto o en viveros, para producir frutos o semillas (Meliponicultura, 2014). La biodiversidad dentro de ecosistemas fomenta mayor biodiversidad de especies de abejas sin aguijón, pues existe una relación entre la evolución de las flores con sus polinizadores nativos (Meliponicultura, 2014).

Asimismo, la aplicación móvil pretende facilitar el acceso a la información antes mencionada, a través de una ficha técnica para los usuarios meta del CINAT dentro de los que se incluye usuarios expertos, meliponicultores, aficionados y estudiantes de carreras afines, con el fin de crear conciencia sobre la importancia, informar y educar sobre la diversidad de especies de abejas que existen en Costa Rica.

Por el lado de las aplicaciones móviles, estas se han convertido en herramientas de gran utilidad en nuestra vida diaria, las cuales permiten por ejemplo la comunicación y el entretenimiento, también están las que facilitan realizar pagos, transacciones o ver el catálogo de productos o servicios y comprarlos (Castillo, 2016). Adicionalmente, hay un cambio en la utilización del teléfono inteligente. En Costa Rica, de acuerdo con el estudio

“Uso del teléfono celular en la población costarricense” del 2017, nos indica que el 94.6% de los encuestados, utiliza el teléfono celular y de ellos el 85,4% lo tiene con pantalla táctil, además el ingreso a redes sociales es un uso frecuente con un 76.8% lo cual nos indica una demanda de internet significativa. (Peña, 2017). Las suscripciones de Internet móvil crecieron 97,5%. Según Sutel 4,8 millones de usuarios están teniendo acceso a la web a través de su celular (Cordero 2018).

Existe otro campo tecnológico denominado Visión por Computadora, este va desde identificaciones biométricas muy comunes en la actualidad, como por ejemplo accesos de seguridad, identificación de personas y capturar gestos como sonrisas en fotografías (Lapedriza, 2012). Este proceso se realiza mediante identificación de patrones y puntos de interés en imágenes, los cuales hacen uso de algoritmos propiamente dedicados a esta labor. También la identificación de objetos a través de una imagen generada por una cámara, este puede verse aplicado tanto en áreas de salud, por ejemplo, para detectar apariciones cancerígenas (Villalobos, 2018), como también en áreas de la biología para la identificación de ciertas especies de abejas como en nuestro caso.

Existe un alto uso de procesamiento de imágenes a nivel mundial distribuidas en una gran cantidad de áreas, otro ejemplo de ello es en el campo de la seguridad. En China, la policía utiliza dispositivos similares a los anteojos de Google para la identificación de potenciales sospechosos (Roussell, 2018). Además de esto, se dice que pueden identificar al sospechoso por la forma en la que camina. *“El reconocimiento del paso es una tecnología que está siendo usada por la policía de Beijing y Shanghai donde pueden identificar individuos incluso cuando sus rostros están oscurecidos o se encuentran de espaldas”* (Roussell, 2018).

En la banca, la visión por computadora es utilizada para hacer depósitos de cheques de forma remota tomando una foto del cheque con su teléfono móvil. El software de visión por computadora en la aplicación captura la foto del cheque, luego verifica si la firma es genuina

(IoT of all, 2017). Otra aplicación que está en desarrollo es la de la búsqueda de montañistas perdidos. Con la ayuda de drones equipados con cámaras además de otros dispositivos, pretenden escanear toda el área de búsqueda para hacer un modelo 3D de la zona y así facilitar la búsqueda de los montañistas. A medida que el dron vuela alrededor de la zona, crea un mapa 3D del terreno. Los algoritmos ayudan a reconocer los sitios no explorados aún. En una estación en tierra se fusionan los diferentes mapas de los diferentes drones en un único mapa que pueden ser monitoreados por los rescatistas (Matheson, 2018).

Vale la pena destacar la importancia de este tipo de proyecto para la escuela de informática de la Universidad Nacional, ya que no solo se permiten fortalecer lazos con otras instituciones internas, si no también incentivar a los estudiantes con distintos tipos de proyectos que fomentan el bien social.

Es proyecto funge como experimento académico, el cual sirve para otras áreas y carreras afines a esta temática, abriendo paso a futuras investigaciones. Adicionalmente se destaca el crecimiento profesional y académico que este proyecto ha proveído junto con la colaboración de profesionales de otras áreas completamente distintas y también la posibilidad de colaborar con una institución que expone su problemática a nivel de apoyo tecnológico.

Para poder lograr el desarrollo de este proyecto, se proponen los objetivos que se describen en el siguiente apartado.

4. Objetivos del Proyecto

4.1 Objetivo general

Facilitar la detección de abejas *Apidae Meliponini* y sus diferentes atributos utilizando algoritmos de visión por computadora mediante dispositivos móviles.

4.2 Objetivos específicos

1. Analizar los métodos de caracterización de abejas *Apidae Meliponini* y sus diferentes atributos mediante algoritmos de visión por computadora.
2. Proponer a partir del análisis realizado, un método de caracterización que utilice el algoritmo de visión por computadora seleccionado.
3. Implementar una herramienta computacional para dispositivos móviles que utilice el algoritmo propuesto.
4. Determinar mediante pruebas de funcionalidad la eficacia de la caracterización de abejas *Apidae Meliponini* con la utilización de la herramienta propuesta.

CAPÍTULO II

MARCO TEÓRICO

CAPÍTULO II: MARCO TEÓRICO

Taxonomía de las abejas Apidae Meliponini

Una tribu es un rango taxonómico que se encuentra entre la familia y el género (BBC, 2014). La tribu *Meliponini* comprende las abejas sin aguijón que viven en las zonas tropicales y subtropicales del mundo (*Michener, 2007*). Las abejas nativas sin aguijón pertenecen a la clase *insecta*, orden *hymenoptera*, familia *apidae*, y a la tribu *Meliponini* (*Michener, 2007*).

En Costa Rica hay 20 géneros y 59 especies, estas abejas son las más comunes y juegan un papel muy importante como polinizadores de la vegetación nativa (*Camargo y Pedro, 2007*). Por otro lado, gran parte de la agricultura nacional depende de estos agentes polinizadores naturales, además son fuente de empleo para muchos meliponicultores, sobre todo por los beneficios que ofrece la miel, producto de las abejas, en múltiples áreas tales como la farmacéutica y gastronomía. Las abejas juegan un papel muy importante tanto a nivel ecológico como industrial y cabe mencionar que Costa Rica no es la excepción, a nivel mundial estos animales tienen un gran impacto de equilibrio en los ecosistemas (*Pacheco, 2018*).

Las *Meliponini* se clasifican de la siguiente manera:

Tabla 1. Clasificación Meliponini

Reino	Filo	Clase	Orden	Suborden	Súper Familia	Familia	Subfamilia	Tribu
Animalia	Arthropoda	Insecta	Hymenoptera	Apocrita	Apoidea	Apidae	Apinae	Meliponini

Fuente: Elaboración propia

A partir de la tribu se encuentra la clasificación por género y especie. La figura siguiente muestra el nivel de aplicación:



Figura 1 . Clasificación Meliponini.

Fuente: Elaboración propia

La taxonomía es, según la Convención en Diversidad Biológica (CBD por sus siglas en inglés) la ciencia de nombrar, describir y clasificar organismos, incluyendo todas las plantas, animales y microorganismos del mundo. Usando observaciones morfológicas, de comportamiento, genéticas y bioquímicas, los taxonomistas identifican, describen y clasifican especies incluyendo las que son nuevas para la ciencia (CBD, 2007). Para clasificar una abeja, los taxonomistas deben verla primero, para así poder describir sus características, tales como su tamaño, color, forma, conducta de las abejas en su piquera, incluso tomar en cuenta si distribución en cuanto a zonas, entre otros.

Visión por computadora

La visión por computadora pretende emular la visión humana para así poder extraer las características de la abeja. La visión por computadora es un subcampo de la inteligencia artificial cuyo propósito es el de programar el computador para que entienda las características de una imagen, más precisamente para deducir las propiedades del mundo tridimensional a partir de imágenes en dos dimensiones (Valdivia, 2016). En general se busca facilitar la extracción de información a partir de los objetos de interés dentro de la imagen a fin de obtener propiedades distintivas para su posterior clasificación (Mansilla 2015).

Para la extracción final de los datos que posee la imagen, es necesario acondicionar la imagen, segmentar o aislar los objetos de interés que se encuentran dentro de la misma y

finalmente extraer características que permitan su clasificación (Mansilla, 2015). Las etapas del procesamiento de imágenes para la extracción de características se detallan a continuación:

- **Captación de la imagen:** Es el método utilizado para obtener la imagen. La imagen se captará por medio de la cámara del dispositivo móvil.
- **Procesamiento de la imagen:** El proceso que sufre la imagen captada al convertirse en digital. Al momento de la captación esta pudo haber sufrido una pérdida de calidad, ruido, desenfoco, etc. Esta actividad de bajo nivel es utilizada primordialmente para reducir el ruido presente en la imagen, mejorar su contraste o resaltar algunos característicos (Mansilla 2015). El propósito del procesamiento es mejorar la calidad de la imagen adquirida eliminando las partes indeseables o realzando las partes de mayor interés en ellas, de esta forma se aumenta la posibilidad de éxito en el trabajo (Valdivia, 2016).
- **Segmentación:** El proceso de segmentación permite dividir una imagen en regiones disjuntas (Mansilla 2015). Es el proceso en el cual se logra descomponer una imagen en sus partes u objetos constituyentes que guarden una fuerte relación con los objetos o el área de interés utilizada en la matriz principal de análisis (Valdivia 2016). Existen varios métodos de segmentación los cuales se detallan a continuación:
 - **Umbralización:** proceso por el cual se convierte una imagen a escala de grises y otra en blanco y negro con la finalidad de que los puntos que sobrepasan el valor del umbral tomen por color blanco y los que no lo pasen se tomen como negro (Valdivia, 2016). Se basa en el agrupamiento de los puntos de la imagen teniendo en cuenta solamente sus características individuales, como nivel de grises, color y textura (Mansilla, 2015).

- **Detección de contornos:** En esta técnica se explota la información de contexto de los píxeles y las estructuras de la imagen, poniendo énfasis en la búsqueda de similitudes entre los puntos o en los cambios significativos de los niveles de gris de los píxeles vecinos (Mansilla, 2015). Es una técnica muy útil para determinar la forma y el tamaño para la clasificación (Valdivia, 2016).
- **Detección de regiones:** implica un agrupamiento y luego la extracción de píxeles similares para formar una región que represente un único objeto en la imagen (Valdivia 2016).

Estas etapas de procesamiento de imágenes se dan gracias a la aplicación de diferentes algoritmos de visión por computadora a los que las imágenes son sometidas. A continuación, se explican algunos de ellos:

Algoritmo SIFT: Es transformación de características invariantes de escala por sus siglas en inglés. El algoritmo SIFT se basa en 4 etapas principales las cuales se listan a continuación:

- **Detección de extremos de espacio de escala:** Hace uso eficiente de la función diferencia de Gaussian (difference-of-Gaussian) para identificar potenciales puntos de interés que son invariantes a escala y orientación (Lowe, 2004). La diferencia de Gaussian es un algoritmo de mejora de imagen en escala de grises que consiste en restar una versión borrosa de una imagen en escala de grises original de otra versión menos borrosa de la original (Davison, 2016).
- **Localización de puntos clave:** En cada ubicación candidata, se ajusta un modelo detallado para determinar la ubicación y la escala. Los puntos clave son seleccionados en base a las medidas de su estabilidad (Lowe, 2004)

- **Asignación de orientación:** Una o más orientaciones son asignadas a cada ubicación de punto clave basado en las direcciones gradientes de imágenes locales (Lowe, 2004).
- **Descriptor de puntos clave:** Los gradientes locales de la imagen son medidos a una escala seleccionada en una región alrededor de cada punto clave. Estos se transforman en una representación que permite niveles significativos de distorsión de la forma local y cambios de iluminación (Lowe, 2004).
- **Emparejamiento de puntos clave:** La mejor coincidencia para cada punto clave es encontrada identificando su vecino más próximo en la base de datos de puntos clave de las imágenes de entrenamiento (Lowe 2004).

Algoritmo SURF: Aceleración de características robustas, de sus siglas en inglés. Se basa en matriz Hessiana por su buen desempeño en tiempo computacional y exactitud (Vangool, 2006).

El descriptor de SURF está basado en propiedades similares a las de SIFT con la complejidad reducida. El primer paso consiste en arreglar la orientación reproducible basada en información de una región circular alrededor de un punto de interés. Luego se construye una región cuadrada alineada la orientación seleccionada y extraer el descriptor SURF de ahí (Vangool, 2006)

Algoritmo FAST: Características por prueba de segmento acelerado, de sus siglas en inglés. El criterio de prueba de segmento opera considerando un círculo de 16 píxeles alrededor de la esquina candidata. (Rosten. Porter. Drummond, 2008)

Algoritmo BRIEF: sus siglas significan Características Primarias Independientes Binarias Robustas. Proporciona un atajo para encontrar las cadenas binarias directamente

sin encontrar descriptores. BRIEF es un descriptor de características, no provee ningún método para encontrar las características. BRIEF es un método más rápido para el cálculo de descriptores de características y coincidencias (Calonder. Lepetit. Strecha. Fua, 2010).

Algoritmo ORB: Es el FAST orientado y BRIEF rotado por sus siglas en inglés. Este algoritmo es una fusión de los detectores de puntos clave FAST y los descriptores BRIEF, con muchas modificaciones (Mordvintsev, 2013)

Librerías de Visión por computadora

A continuación, se describen algunas librerías de visión por computadora que actualmente son usadas en los diferentes campos.

- **OpenCV:** OpenCV es una librería de código abierto que incluye cientos de algoritmos de visión por computadora. Se puede hacer uso de esta en C++, C, Python y Java con soporte para Windows, Linux, MAC OS, iOS y Android. Lanzada bajo licencia BSD gratuita para uso académico y comercial.
- **BoofCV:** De acuerdo con su sitio web (<http://boofcv.org>), es una librería de código abierto escrita desde cero para visión por computadora en tiempo real. Su funcionalidad cubre un rango de temas: procesamiento de imágenes de bajo nivel, calibración de cámara, detección y rastreo de características. Se lanzó bajo licencia Apache 2.0 para uso comercial y académico.
- **Nasa Vision Workbench:** según su sitio web (<https://software.nasa.gov/software/ARC-15761-1A>) es un *framework* modular, extensible y multiplataforma, que soporta un rango de tareas, incluidas el análisis automatizado de ciencia e ingeniería, procesamiento de imágenes satelitales y reconstrucción de ambientes 2D y 3D

La visión por computadora es muy utilizada y se aplica en diferentes áreas. Algunas de ellas se mencionan a continuación (Szeliski, 2010):

- **Imágenes médicas:** por ejemplo, estudios a largo plazo de la morfología del cerebro de las personas a medida que pasan los años.
- **Inspección de máquinas:** utilizada para aseguramiento de la calidad para medir las tolerancias de las alas de aviones.
- **Reconocimiento óptico de caracteres:** Reconocimiento de placas de automóviles. Lectura de códigos postales escritos a mano.
- **Vigilancia:** detección de intrusos en lugares privados, análisis del tráfico en carretera.
- **Seguridad automovilística:** detección de obstáculos inesperados en la carretera como por ejemplo los peatones en condiciones donde radares no funcionan.

Dentro de la visión por computadora, al ser subcampo en la inteligencia artificial, abarca también los siguientes conceptos:

- **Deep Learning:** El *Deep learning* es un subcampo de *machine learning*, así como el machine learning lo es de la inteligencia artificial. La diferencia radica en el tratamiento de los algoritmos, ya que el Deep Learning permite a las computadoras aprender de la forma más cercana a como lo realizan los humanos por medio de redes neuronales (Gorini 2019).

- **Red Neuronal Convolutacional:** De sus siglas en inglés CNN, permiten la capacidad de un ordenador de identificar y procesar imágenes a partir de un entrenamiento supervisado, de esta manera identificar de manera automática diferentes tipos de patrones y así ejercer diferentes acciones, por ejemplo, la identificación de anomalías en el cuerpo humano, como tumores, conducción autónoma de autos, predicciones en economía y finanzas, entre otras. Dentro de su procesamiento, este se lleva a cabo mediante una jerarquía, la cual cada nivel detecta diferentes puntos de interés, por ejemplo, primeramente líneas, seguidamente curvas, luego siluetas y finalmente rostros, buscando de esta manera cada vez la especialización de un objeto. Adicionalmente su nombre “neuronal” es debido a que este tipo de algoritmo se asemeja a la manera como los humanos aprenden y específicamente en este subcampo de la inteligencia artificial, cada neurona es el producto del área en píxeles de una imagen y con respecto a “convolutacional” haciendo referencia a los grupos de píxeles cercanos que serán procesados a través de las diferentes fórmulas matemáticas contra una imagen u objeto buscado utilizado como base (Bagnato 2018).

Sistemas similares

Como parte de los sistemas similares identificados se encontró el ***Sistema Automático de clasificación de abejas sin aguijón (Apidae Meliponini) basado en el contorno y venación de sus alas*** realizado por El Instituto Tecnológico de Costa Rica para identificar, de acuerdo con taxonomía y mediante un sistema automático, la especie a la cual pertenece una abeja. Este sistema identifica abejas por medio de imágenes tomadas en un entorno de eliminación de variables y parametrizaciones para determinar las correspondencias, el porcentaje de éxito logrado por este sistema fue de un 86.5, pero al incluir factores como el género logró el 97.52%. Esto los llevó a la conclusión que incluir más factores y variables de discriminación, ayudan a una mejor clasificación de las abejas dentro de su especie (Prendas, 2015).

La fundación Árboles Mágicos ha creado y promueve una aplicación para la identificación de árboles denominada **Ojeadores**, la cual utiliza diferentes insumos y variables para poder dar como resultado la especie del árbol ante la cual un trabajador de campo o estudiante se encuentra. Dicha aplicación mantiene ciertas restricciones de uso que se obtienen mediante un pago o compra dentro de la misma, de esta forma la organización obtiene algunos ingresos para financiarse. Esta aplicación maneja un concepto similar al que se quiere implementar para la identificación de abejas, por ejemplo, al permitir discriminar mediante parámetros como la zona, el color o nombre y dando como resultado, la información de la especie (*Apiary Book Ltd., 2018*).

Apiary Book es una aplicación móvil que permite dar seguimiento a varios factores de interés de las abejas, por ejemplo, la cantidad de grupos, salud, mantenimiento, tratamientos, información de las familias de abejas, inventarios, entre otras funcionalidades de producción para las granjas. Este sistema móvil es de suma utilidad para todos aquellos productores de miel, meliponicultores o técnicos apícolas que lleven a cabo diferentes estudios de campo, este brinda una útil plataforma para llevar a cabo trabajos de seguimiento, producción y control. Por otro lado, su interfaz la hace una herramienta fácil de utilizar (*Árboles Mágicos, 2016*). Cabe mencionar que la aplicación no realiza la identificación de abejas, aunque permite llevar el registro de estas.

Otro ejemplo es la aplicación ***Deep Learning in openCV for visually impaired***, la cual tiene como objetivo demostrar los recursos enfocados en openCV en conjunto a la técnica de red convolucional neutral para identificar y narrar los objetos presentes en el entorno enfrente de la cámara (de Oliveira, 2018).

Áreas de conocimiento

- **Aplicaciones móviles**

Se denomina aplicación móvil o *app* toda aplicación informática diseñada para ser ejecutada en teléfonos inteligentes, tabletas y otros dispositivos móviles. Por lo general se encuentran disponibles a través de plataformas de distribución, operadas por las compañías propietarias de los sistemas operativos móviles como Android, iOS, BlackBerry OS y Windows Phone, entre otros (Santiago & Trbaldo & Fernández, 2015).

La creación de una aplicación móvil hará mucho más sencillo el trabajo de cargar consigo la herramienta ya sea para fines técnicos o académicos, ya que bastará contar con un dispositivo móvil basado en Android, para poder utilizar la aplicación que se propone en este proyecto.

- **Arquitectura cliente-servidor**

Una arquitectura cliente servidor es una comunicación que se ejecuta en diferentes máquinas, en donde el servidor funciona como un proveedor de servicios mientras que el cliente llegaría a ser el consumidor de dichos servicios, cabe mencionar que puede existir múltiples clientes consumiendo dichos servicios. El servidor y el cliente se comunican mediante un mecanismo de envío y recepción de mensajes, por ejemplo a través de un API, donde el cliente realiza una petición y el servidor responde (Vale & Gutierrez, 2005).

Dentro de las características y ventajas que existen dentro de este tipo de arquitecturas de sistemas informáticos, es que ambas entidades (cliente y servidor) pueden a su vez actuar como si fueran una sola máquina pero también por separado, permitiendo una mayor eficiencia al no depender de los mismos recursos a nivel de *hardware* con los que se cuenta. Por otro lado, no es necesario que ambos entes se estén ejecutando sobre la misma plataforma, dicho de otra manera, puede existir y mantener la comunicación inclusive desde diferentes dispositivos de diferentes tipos, por ejemplo, computadoras, servidores, dispositivos móviles, laptops, tabletas, entre otros (Vale & Gutierrez, 2005).

Como se menciona anteriormente, una gran ventaja es que varios clientes pueden realizar consultas o peticiones al servidor de manera simultánea, lo que separa las tareas y evita el embotellamiento de procesos si todo fuera ejecutado en la misma plataforma. Adicionalmente y similar al caso de los clientes, se puede contar con un mismo punto de recursos donde múltiples servidores funcionen como uno solo, siendo transparente también para el cliente o los clientes que realicen las consultas (Vale & Gutierrez, 2005).

En este proyecto se considera este tipo de arquitectura ya que no se sobrecargará todo el trabajo de análisis, procesamiento y almacenamiento de la información dentro de un dispositivo móvil funcionando como cliente, además de las limitaciones a nivel de hardware que puedan existir, demandando por ejemplo una mayor exigencia del dispositivo en cuando a batería, capacidad de realizar tareas simultáneas, procesamiento de imágenes y video, agotamiento de recursos de memoria y velocidad de respuesta. Por otro lado, al separar el sistema bajo esta arquitectura, permitirá que cualquier futura ampliación pueda adaptarse transparentemente con su implementación en el servidor y hacer uso de los servicios que ofrece.

○ **RESTful API**

De sus siglas in inglés, “*Application Program Interface*”, es una serie de reglas que permiten o habilitan la comunicación entre diferentes programas, de manera que habilita a un programador desarrollar una comunicación entre un servidor y un cliente. Esta interfaz de comunicación, fue introducida por Roy Fielding en el año 2000, en donde se define como un servicio de comunicación entre plataformas (Naeem, 2021).

Gracias a la integración de un API, cuando un cliente requiere hacer una consulta con respecto a cierta información, dicha consulta es permitida gracias al API creado por un desarrollador, de manera que el servidor al recibir y detectar el tipo de consulta puede devolver una respuesta con respecto a los insumos recibidos, esto gracias a la integración nuevamente, del API (Naeem, 2021).

Debido a que la implementación de REST soporta y permite la comunicación a través del protocolo HTTP, el desarrollador diseña la estructura del API de manera tal que por medio de métodos descritos por el protocolo en el RFC 2616, se puedan realizar las diferentes consultas hacia el servidor por medio de los métodos GET, PUT, POST y DELETE, que posteriormente permiten extraer información, alterar o actualizar el estado de un objeto, crear nuevas entradas de información y eliminar información respectivamente. (Naeem, 2021).

- **Framework de programación**

Se definen como *frameworks* de programación a la estructura de reutilización de código implementadas debido a la alta complejidad del desarrollo de las aplicaciones y programas actuales. (Ortega & Guevara & Benavides, 2017).

Dichos frameworks de desarrollo se encuentran actualmente en múltiples lenguajes de programación, por ello, la elaboración de dichos frameworks requiere una experticia adecuada. Por otro lado, en la actualidad, los desarrolladores de software solventan problemas informáticos y soluciones específicas, debido a esto, se enfrentan a diferentes desafíos como por ejemplo, la de evitar re-trabajo y aprovechar la reutilización de código, que vaya además acorde a las diferentes etapas dentro del ciclo de desarrollo en donde no se pretenda invertir más tiempo de lo necesario. La problemática también se encuentra cada vez que surgen nuevos requisitos. Ahora, gracias a la implementación de frameworks de programación, se pueden aprovechar esfuerzos anteriormente realizados con respecto a diseño y programación para mitigar las situaciones o problemas anteriores. Esencialmente los *frameworks* facilitan la reutilización de código y el desarrollo más rápido de aplicaciones, pero esta siempre va de la mano con la manera en que se utilice el *framework*, lo que demanda también conocimiento y preparación por parte del programador. (Ortega & Guevara & Benavides, 2017).

Alcance

El proyecto contempla la creación de una aplicación móvil que facilite la información e identificación de la especie a la que pertenece una abeja, de esta forma usuarios expertos, meliponicultores, aficionados o estudiantes interesados en esta área puedan utilizarla y realizar investigaciones de campo. La aplicación permitirá ya sea identificar la especie a la que pertenece una abeja de la tribu *Meliponini* o discriminar y acercar al usuario dentro del rango más cercano posible. Las pruebas y desarrollo inicial se harán solamente para una de las especies, de esta forma verificar y validar el algoritmo de identificación.

Los insumos necesarios para poder realizar dicha discriminación, se llevará a cabo a partir de investigaciones, reuniones y entrevistas con los expertos del CINAT, permitiendo así una comprensión más amplia y consciente del área en la que se trabaja, para finalmente transmitir todo este conocimiento y forma de identificación de abejas en la aplicación objetivo a desarrollar

Se pretende programar una base de datos con los insumos necesarios, por medio de la aplicación móvil y la implementación de un algoritmo de identificación, para determinar a cuál especie pertenece o dentro de cual ámbito de especies se encuentra una abeja. Se comprenden únicamente las especies del territorio nacional costarricense.

Cabe mencionar que se utilizará el SDK de OpenCV como núcleo para el procesamiento de imágenes, el cual brinda más de una opción y serie de métodos, de código abierto, que permiten realizar los diferentes pasos para la caracterización pertinente de una imagen, como lo son la carga de la imagen, asignación de grises, obtención y modificación de píxeles, segmentación, obtención del contorno, área y perímetro además de su almacenamiento y procesamiento.

Como se ha mencionado, la aplicación podrá ser ejecutada únicamente en dispositivos móviles compatibles con el sistema operativo Android, de esta forma no se incurrirá en gastos monetarios por la adquisición de licencias extra para su creación debido al tipo de herramientas de software necesarias para su programación.

CAPÍTULO III

METODOLOGÍA

CAPÍTULO III: METODOLOGÍA

A continuación, se presenta el conjunto de actividades a realizar para alcanzar los objetivos planteados.

1. Tipo de investigación

Este trabajo se categoriza como una investigación aplicada y exploratoria, ya que posee propiedades tanto para encontrar estrategias, en este caso un algoritmo utilizado por medio de una aplicación móvil, que permita solventar un problema específico, el cual es identificar especies de abejas *Apidae Meliponini* por medio de la captación de imágenes. Por otro lado, a nivel exploratorio ya que es un tema o práctica la cual no hay evidencia de que existan sistemas iguales que puedan realizar dicha identificación de especies de abejas por medio de visión por computadora. Posteriormente, se pretende realizar la recolección de datos basado en las pruebas y poder brindar conclusiones y recomendaciones que permitan la continuidad de futuras investigaciones.

Se considera fundamental el conocimiento de los procesos de clasificación taxonómica utilizado para las abejas. Para la obtención de este, se realizaron las reuniones necesarias con Ingrid Monge y Mario Gallardo, expertos en materia del CINAT. Además, se investigará la tecnología propuesta y asociada a la visión por computadora. La información obtenida se complementará con información disponible en diferentes medios bibliográficos y la web. Además, dentro de esta etapa, se contempla la investigación referente al desarrollo de aplicaciones móviles. Adicionalmente, la investigación de los diferentes algoritmos para la identificación de objetos, esto a través del procesamiento de imágenes por medio del SDK de OpenCV.

2. Población y muestra

Como parte de la población y muestra, se contemplan únicamente a la tribu *Apidae Meliponini* dentro del territorio costarricense. Específicamente se utilizarán las siguientes especies dentro de la tribu mencionada como objetos de prueba:

- *Melipona Costaricensis*
- *Melipona Fallax*
- *Tetragona Ziegleri*
- *Oxitrigona Mellicolor*
- *Trigona Nigerrima*
- *Trigona Silvestriana*
- *Trigona Corvina*
- *Partamona orizabaensis*

Se escogieron las anteriores especies, ya que brindan características similares y distintas entre sí, lo que permite una mayor variabilidad de resultados debido a que si se comparan especies muy similares en cuanto a características físicas, se podrá medir el potencial del algoritmo propuesto para discernir entre especies muy parecidas, además de medir a un menor nivel también la probabilidad de identificar especies completamente distintas. Debido a ello, se seleccionaron especies dentro del mismo género, como en el caso de las *trigonas*, las cuales también comparten similitud con respecto a la *Partamona orizabaensis*. Similarmente, se toma en cuenta dentro del mismo género a las *Meliponas*, tanto *Fallax* como la especie originaria de Costa Rica, *costaricensis*. Finalmente, aleatoriamente se escogieron otras especies con el fin de completar y obtener diversos resultados, para poder observar distintos comportamientos del algoritmo y aplicación móvil.

3. Descripción de instrumentos

Para medir la eficacia de los algoritmos que provee OpenCV como motor de análisis de imágenes, se pretende tabular todas las pruebas realizadas en diferentes tablas con el fin de comparar entre sí resultados y concluir cuales muestran mejores porcentajes de éxito en discriminar o descartar una especie Apidae Meliponini, dentro de un grupo de abejas de la misma tribu y hasta género.

Adicionalmente, también se espera medir los porcentajes de éxito en la identificación de la especie de abeja ante la que nos encontramos o estamos solicitando analizar por medio de la aplicación. Cabe mencionar que el recopilar los datos de esta manera permite realizar una pausa en cuestión de análisis de resultados, cambio de variables o entorno de pruebas, y realizar así de nuevo otras fases e iteraciones de verificaciones, con el fin de mejorar u obtener conclusiones relevantes con respecto a los resultados obtenidos en cada fase. Una vez tabulados los datos, se resumen y se comparan, permitiendo finalmente proponer un algoritmo en conjunto con la aplicación.

4. Procedimientos para analizar la información del diagnóstico

Como parte del procedimiento que se propone utilizar, se seguirá una tendencia analítica descriptiva, ya que a pesar de que se utilizará OpenCV como motor de procesamiento de imágenes, se investigarán sistemas similares y diferentes tendencias en cuanto al desarrollo de aplicaciones que utilicen visión por computadora, lenguajes de programación disponibles en esta tecnología y arquitectura de sistema. Además, se incluye

una recopilación de datos y resultados relacionados a las pruebas, para que de esta forma puedan ser tabulados y analizar resultados y recomendaciones debido a la naturaleza de la investigación aplicada y exploratoria de este proyecto.

Cabe mencionar que se realizarán consultas y se buscará asesoramiento de parte de otros colegas dentro del campo informático, con el fin de obtener comentarios, retroalimentación y observaciones sobre las mejores tecnologías y arquitectura a implementar actualmente para este proyecto. Esto ayudará a contar con fuentes tanto de documentación como de experiencia en el área de desarrollo de aplicaciones móviles, que permitan una implementación veraz y sencilla de utilizar por parte del usuario final del sistema como un todo.



Figura 2. Flujo Propuesta

Fuente: Elaboración propia

CAPÍTULO IV

PROPUESTA DE SOLUCIÓN

CAPÍTULO IV: PROPUESTA DE SOLUCIÓN

1. Propuesta de solución

Como punto de partida, se llevó a cabo una entrevista inicial con el CINAT, con el propósito de comprender la problemática y limitaciones que actualmente presenta el instituto. Como se mencionó en el Capítulo I, apartado 2, hay una falta de apoyo tecnológico para digitalizar y perdurar la información que cuentan para la identificación de especies de abejas *Apidae Meliponini*. Debido a eso y aunado a la importancia de las abejas como agentes polinizadores, la producción de miel y la conservación de especies, la información y llaves de identificación que actualmente utilizan se queda en papel, lo que provoca no poder promover de una manera activa los recursos y conocimientos con los que sí cuentan, lo que recalca una falta de interés en el tema por parte del público, consecuentemente la disminución de expertos en el área y la posibilidad de automatizar procesos de identificación de especies de abejas que actualmente realizan de manera manual.

Una vez recibida la información acerca de la problemática y detalles, se procede a analizar dicha recopilación con el fin de idear una manera en la que el instituto obtenga apoyo tecnológico y adicionalmente, generar una nueva herramienta informática. Para esto y en concordancia con el CINAT, se propone una aplicación móvil, debido a su portabilidad y al auge en cuanto al uso de esta plataforma hoy en día, que permitiría una fácil distribución y uso. Adicionalmente, para aprovechar la captación de imágenes que tiene esta plataforma para realizar el análisis e identificación de especies de abejas de manera automática. Dicho sea de paso, se procedería primeramente a realizar una investigación en cuanto a las tecnologías referentes a visión por computadora.

Antes de indagar en este campo, se analiza la arquitectura del sistema como un todo, y gracias a las bondades que ofrece se propone utilizar una arquitectura cliente-servidor, la cual se ampliará más adelante. En paralelo también se investigarán y analizarán las posibilidades en cuanto a frameworks de desarrollo para dispositivos móviles.

Mencionadas las dos tecnologías anteriores a nivel de arquitectura, la propuesta del proyecto incluye tanto la aplicación móvil, como el sistema gestor para su debido mantenimiento, por lo que se procederá al desarrollo de ambas.

Una vez concluida la etapa de desarrollo de la aplicación móvil y el sistema gestor, además de la integración con la tecnología de visión por computadora, se procede a realizar pruebas funcionales y a nivel de sistema para verificar que efectivamente la aplicación y sistema gestor realizan las tareas para las cuales fueron hechos y además otra fase de pruebas relacionadas con la efectividad de la aplicación para identificar especies de abejas.

Después de las fases de pruebas, se recolectan todos los resultados de las diferentes fases y métodos probados y se analizan dichos resultados con la finalidad de visualizar y proponer cuál algoritmo, con base en los resultados, presenta mejores características tanto de aserción de una especie como de discriminación o descarte de una especie dentro de un grupo dado.

Para finalizar, se provee una propuesta de algoritmo a utilizar, el sistema gestor y la aplicación móvil, proporcionando la documentación y el manual de usuario pertinente para el debido uso.

A continuación, se detallan las tecnologías que se propone utilizar:

- **Django**

Creado por Adrian Holovaty, Simon Willison, Jacob Kaplan-Moss y Wilson Miner, es un *framework* de software libre para el desarrollo web de alto nivel, el

cual permite una rápida y limpia implementación pragmática, evitando problemas comunes a la hora realizar implementaciones web, permitiendo al usuario enfocarse en el desarrollo puro de la aplicación meta. Ejecutándose del lado del backend, dentro de sus características se encuentra que es muy rápido de desplegar y comenzar a utilizar, ya que fue diseñado de manera tal que ayude a los desarrolladores a llevar la aplicación del concepto a su completitud lo más rápido posible. Además, se considera un *framework* sumamente seguro, toma muy en serio este aspecto y ayuda a los desarrolladores a evitar errores que comúnmente suceden durante la creación de una aplicación y sus comunes vulnerabilidades. Es sumamente escalable, toma el mejor aprovechamiento de las características del hardware y permite una segregación muy limpia en cuando a la implementación de la capa de base de datos y la capa de aplicación. (*Django*, 2021)

- **Python**

Creado por Guido van Rossum, es un lenguaje de programación que puede ser implementado con base en diferentes paradigmas de programación. Además, es un lenguaje interpretado, en donde uno de sus mayores aspectos positivos, y donde se hace mayor énfasis, es en la facilidad de su comprensión, debido a su limpia sintaxis y semántica. Por otro lado, incentiva de gran manera a la re-utilización de código, al implementar un esquema de soporte de módulos y paquetes previamente elaborados. (*Python*, 2021)

Se le llama un lenguaje multiparadigma ya que soporta los siguientes paradigmas:

- Funcional: Basado en el uso de argumentos y funciones, donde hay un uso elemental de la recursividad y funciones anidadas (Vaca, 2011).
- Imperativo: Funciona de manera lineal, ejecutando cada línea de código en el orden que fue escrito (Vaca, 2011).

- Orientado a objetos: Basado en la instanciación de objetos más que sobre la ejecución de acciones, en donde cada objeto tiene su tipo, atributos y puede ser utilizado en función a relaciones con otros objetos (Vaca, 2011).

- **Ionic**

Creado por Max Lynch, Ben Sperry y Adam Bradley, es un equipo de desarrollo de software de código abierto para el desarrollo de aplicaciones híbridas, desde aplicaciones móviles, aplicaciones web y sitios web progresivos. Se considera su utilidad ya que con base en el mismo código fuente, puede generar los diferentes tipos de aplicaciones para diferentes tipos de plataformas como por ejemplo Android o iOS. Posee componentes previamente diseñados para facilitar la implementación a nivel de interfaz de usuario. Además, permite realizar una implementación indiferentemente de la tecnología a nivel de lenguaje de usuario que el programador prefiera o domine mejor, como por ejemplo Angular, React, Vue o sin frameworks del todo con la utilización de vanilla Javascript. Cabe destacar que posee una documentación amplia y fácil de acceder. (*Ionic*, 2021).

- **Git**

Diseñado por Linus Torvalds y Junio Hamano, es un software utilizado para el control de versiones, enfocado en la confiabilidad, compatibilidad y trazabilidad para las versiones de un sistema o aplicación. Tiene un manejo de “*branching*” muy eficiente, al permitir diferentes versiones del mismo código, pero sin afectar las versiones estables, de esta manera permite identificar posibles problemas a lo largo del desarrollo de

una aplicación o programa y, seguidamente, detectar en cuál versión se encuentra el problema, realizando un “rollback” al estado anterior del código.

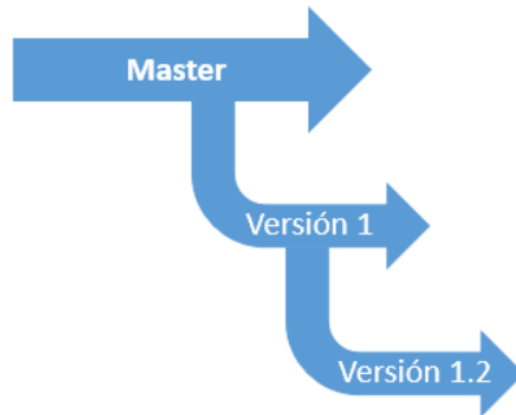


Figura 3 Ramificación de ejemplo git

Fuente: Elaboración propia

- **OpenCV**

Creado por Intel, y como sus siglas lo indican “*Open Computer Vision*”, es una librería de software libre que provee una infraestructura para la creación de aplicaciones de visión artificial y machine learning. La librería posee más de 2500 algoritmos optimizados, los cuales incluyen una comprensible ejemplificación de los diferentes algoritmos que permiten realizar tareas como por ejemplo: identificar rostros, objetos, clasificar acciones humanas a partir de video, producir imágenes de alta calidad, encontrar imágenes similares a partir de una colección dada, entre otros. Posee una comunidad de más de cuarenta y siete mil de usuarios y actualmente excede más de dieciocho millones de descargas. Esta librería es extensamente utilizada en su actualidad por diferentes compañías, grupos de investigación y gobiernos. Una gran ventaja que ofrece es que puede ser utilizable a partir de diferentes lenguajes de programación, como Python

C++, Java y MATLAB y tiene soporte para distintos sistemas operativos como Windows, Linux, Android y Mac OS. (Ionic, 2021).

A continuación, se detallan los elementos de la arquitectura del sistema a utilizar:

Modelo cliente-servidor

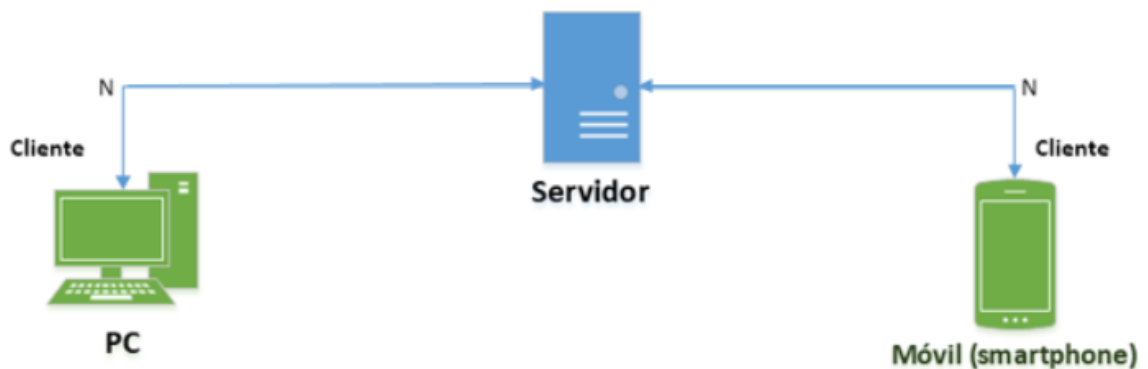


Figura 4 . Modelo Cliente-Servidor

Fuente: Elaboración propia

Con base en la figura anterior, se propone un modelo cliente servidor, de manera que se tiene uno o varios servidores, indicado por la letra N, trabajando de manera centralizada, atendiendo las diferentes peticiones que realizan los clientes dentro del sistema. Cabe destacar que puede haber múltiples clientes realizando peticiones desde diferentes plataformas, lo cual no es una limitante para el sistema y permite la utilización de la aplicación meta por varios usuarios simultáneamente. También, si se realizan consultas con respecto al análisis de imágenes desde una PC, se cargarían imágenes de abejas a partir de archivos ubicados en los directorios del sistema operativo, ya que la webcam no sería identificada como nativa dentro del sistema. Caso contrario a la plataforma móvil, donde se tiene una cámara integrada y nativamente Android detecta dicho hardware permitiendo así, por medio de la aplicación, tomar directamente fotografías de abejas y enviarlas para análisis al servidor. Finalmente, el servidor enviará una respuesta, luego del análisis y

solicitud, a cada cliente respectivamente, sean tareas de mantenimiento (agregar, consultar, actualizar o eliminar abejas o usuarios) o también análisis de imágenes de abejas enviadas.

Sistema gestor

Se propone crear un sistema gestor, el cual facilite y permita al administrador del sistema realizar el mantenimiento pertinente al sistema, ya sea agregar, listar, actualizar o eliminar tanto usuarios como abejas dentro de la base de datos del sistema. Además de otras funcionalidades útiles para el usuario final tales como filtrar búsquedas en una lista de abejas o usuarios, generar imágenes dentro de la base de datos para usos posteriores de manera manual o inclusive realizar análisis de imágenes de abejas para saber qué especie es.

Aplicación Móvil

Como parte de la solución y alineado con los objetivos, se propone crear una aplicación móvil la cual permita realizar las consultas hacia el servidor por medio de imágenes capturadas por la cámara o en caso de carencia de la misma, permite cargar imágenes de abejas previamente tomadas y guardadas en el carrete, obteniendo como respuesta una ficha técnica con respecto a la entrada dada, en caso contrario, no se obtendrán resultados, ya que la abeja no fue encontrada. Desde la misma aplicación móvil, si se cuenta con acceso y privilegios de administrador, también se pueden realizar tareas de mantenimiento al sistema.

OpenCV-Template Matching y Cascade Classifier

Como parte de la investigación realizada y asesoramiento, se utilizó OpenCV como “motor” para el análisis de las imágenes, ya que las opciones que ofrece esta librería *Template Matching* y *Cascade Classifier* se adaptan para realizar la identificación de objetos, en este caso en específico, de abejas.

Template Matching

Es un método para la búsqueda y localización de una imagen usada como plantilla o entrada dentro de otra más grande. OpenCV incluye su propia función para este propósito en donde antepone y analiza la imagen de entrada sobre la imagen más grande y a través de múltiples métodos de comparación implementados en OpenCV devuelve en escala de grises donde exactamente cada pixel denota un mayor acercamiento con respecto a la imagen de mayor tamaño (*OpenCV, 2021*) .



Figura 5 . Ejemplificación Template Matching

Fuente: Obtenido de OpenCV.org el 31 de marzo del 2021.

Como parte del proceso de identificación, se realiza una captación primeramente de la imagen de entrada o plantilla y luego de la imagen base o mayor. Con esto se procesa la imagen, ya sea por medio de una toma de fotografía en vivo o la carga desde una colección de imágenes. Seguidamente, se lleva a cabo el procesamiento de la imagen, el cual consiste en su propia digitalización, el cual dicha imagen puede sufrir ciertas pérdidas de calidad debido al ruido o desenfoque debido a las limitaciones a nivel de hardware. El procesamiento es utilizado para realizar trabajos a bajo nivel y amortiguar dichas carencias debido al ruido, mejorar su contraste y resaltar características con la finalidad de lograr la mejor calidad posible y aumentar la posibilidad de éxito de identificación (*OpenCV, 2021*).

Posteriormente, se realizan trabajos de segmentación, donde la finalidad consiste en buscar los mismos patrones en cuanto a píxeles en la imagen mayor con respecto a la imagen de entrada. Como parte de este proceso de segmentación, se incluye realizar el trabajo de Umbralización, el cual se convierte la imagen a escala de grises y otra en blanco y negro, con la finalidad de visualizar el agrupamiento de los puntos de interés con respecto a características individuales, nivel de grises o tonos, nivel de color y textura (*OpenCV, 2021*).

Finalmente se realiza la detección de contornos y regiones, en donde se buscan los cambios significativos en cuanto a niveles de grises entre píxeles y sus vecinos, agruparlos y extraer dichos grupos similares con la finalidad de encontrar similitudes formando una región que representa un único objeto de la imagen que, en el caso anterior, sería el rostro.

Template matching tiene 6 métodos disponibles (*OpenCV, 2021*):

- TM_SQDIFF
- TM_SQDIFF_NORMED
- TM_CCORR
- TM_CCORR_NORMED
- TM_CCOEFF
- TM_CCOEFF_NORMED

Estos métodos se utilizan para realizar los diferentes cálculos y procesos mencionados anteriormente, ya que cada uno posee una fórmula matemática distinta. De esta forma, se pueden obtener variabilidad de resultados e incluso, con base en los resultados, definir cuál es mejor o cual tiene un mayor nivel de aserción y o acercamiento con respecto a una imagen de entrada sobre la imagen base o mayor, para efectos prácticos y en este caso, especies de abejas (*OpenCV, 2021*).

Cascade Classifier

La detección de objetos usando la característica del “Haar Cascade” es una opción efectiva para la detección de objetos propuesta por Paul Viola and Michael Jones. Es un

enfoque basado en machine learning donde una función en cascada es entrenada como un clasificador con muchas imágenes positivas y negativas, posteriormente se logra utilizar dicho clasificador para detectar imágenes dentro de otras (*OpenCV, 2021*).

Dentro de este enfoque, Haar hace utilización de las características presentes dentro de una imagen, de manera que funcione como un motor convolucional. Por ejemplo, cada característica captada es un valor obtenido sustrayendo la suma de píxeles bajo un rectángulo blanco y la suma de píxeles bajo un rectángulo negro (*OpenCV, 2021*).

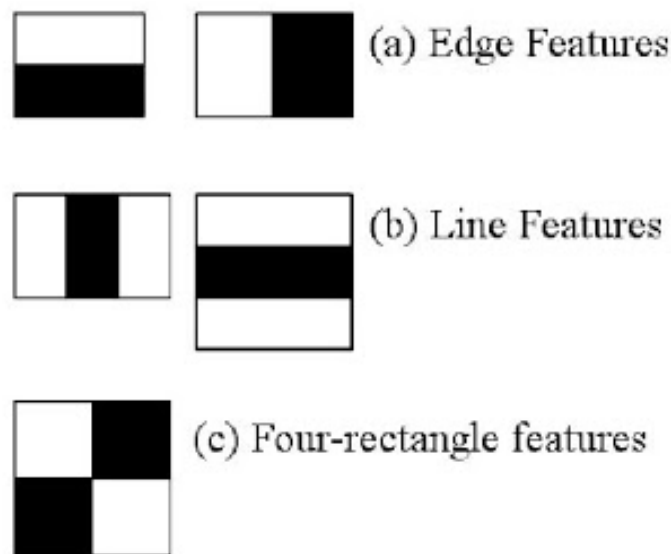


Figura 6 . Ejemplificación Cascade Classifier

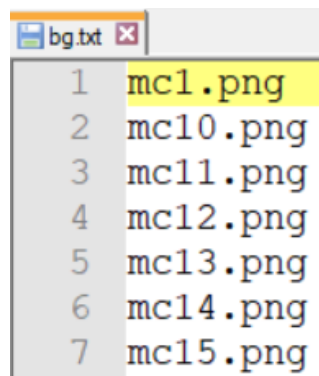
Fuente: Obtenido de OpenCV.org el 31 de marzo del 2021.

Como parte del proceso de entrenamiento del algoritmo se procede mediante dos fases principales:

Preparación de la información para el entrenamiento: en esta fase se realiza una separación y carga entre lo que son las imágenes negativas y positivas. Entiéndase como imágenes negativas, todas aquellas que no son el objeto meta a identificar, en este caso, todas las imágenes menos la que queremos identificar mediante el entrenamiento. Por otro

lado, las imágenes positivas se entienden por todas aquellas que si son el objeto meta, siguiendo la misma línea, todas aquellas imágenes que si son referentes a la especie que queremos identificar. Ahora, una vez listas dichas imágenes, las cuales se recomiendan en una cantidad grande, por ejemplo por lo menos 30 positivas y más de 100 negativas, con el propósito de tener mejores resultados, ya que se daría un mayor entrenamiento. Con las imágenes listas se procede a elaborar un documento centralizado, tanto para imágenes negativas como positivas.

En el caso de las imágenes negativas, simplemente se tiene que incluir en un archivo de texto, línea por línea y en orden, el nombre de cada imagen, seguido de su ubicación. Por lo general, la ubicación de todas las imágenes negativas estaría dentro del mismo folder. Por ejemplo, las imágenes se agruparían en un archivo llamado <nombre>.txt como se muestra a continuación:

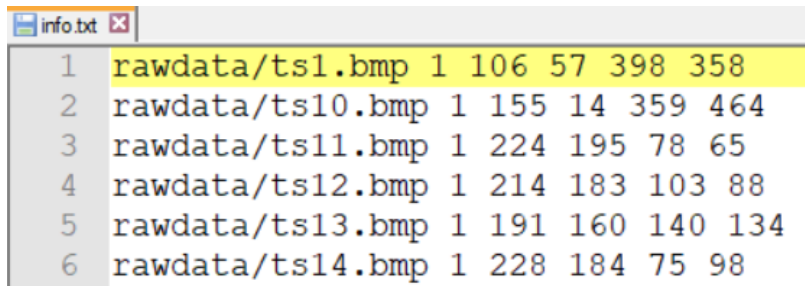


```
bg.txt
1 mc1.png
2 mc10.png
3 mc11.png
4 mc12.png
5 mc13.png
6 mc14.png
7 mc15.png
```

Figura 7. Ejemplo de centralización de imágenes negativas

Fuente: Elaboración propia.

En el caso de las imágenes positivas, seguiría una idea similar, pero adicionalmente se segmentaría la imagen por medio de OpenCV o cualquier otra herramienta que permita hacerlo y que indique las coordenadas exactas del objeto meta a ser identificado. En el archivo final se visualizarían de la siguiente manera:



```
1 rawdata/ts1.bmp 1 106 57 398 358
2 rawdata/ts10.bmp 1 155 14 359 464
3 rawdata/ts11.bmp 1 224 195 78 65
4 rawdata/ts12.bmp 1 214 183 103 88
5 rawdata/ts13.bmp 1 191 160 140 134
6 rawdata/ts14.bmp 1 228 184 75 98
```

Figura 8. Ejemplo de centralización de imágenes positivas

Fuente: Elaboración propia.

Nótese el orden que llevaría, el cual es el nombre de la imagen en formato .bmp seguido de las coordenadas indicando el objeto a ser identificado dentro de la totalidad de la imagen.

Posteriormente, para las imágenes positivas, a pesar de que se recomienda tener una mayor variabilidad de las mismas, se podrían generar diferentes plantillas con base en una misma imagen, pero no daría los mismos resultados, ya que se parte de una sola imagen, siendo esta más estática, con poca variabilidad, mientras que si se incluyen más imágenes positivas del mismo objeto meta, se pueden obtener mejores resultados, ya que el objeto, podría ser captado desde diferentes ángulos, fondos, iluminación, acercamiento, entre otros.

Con las imágenes una vez listadas en los archivos previamente mencionados, se generan los llamados vectores, los cuales son archivos que contienen las anotaciones combinando la información tanto de las imágenes positivas como negativas, coordenadas y nombres. Dichos vectores serán utilizados para iniciar el proceso de entrenamiento, en donde se pueden indicar distintos parámetros previos a iniciar el proceso, por ejemplo, unos de los más comunes:

- -bg, background description file
- -num, número de imágenes de muestra a generar
- -w, ancho en píxeles de la imagen de salida

- -h, alto en píxeles de la imagen de salida

El producto final luego del proceso de entrenamiento, el cual consiste en la ejecución de un proceso proveído por OpenCV con los parámetros anteriores, es un conjunto de elementos denominados *cascades*, los cuales son archivos que representan y contienen la información con las etapas con en las que una imagen será comparada para saber si nos encontramos en presencia del objeto meta o no. Para esto el proceso que se ejecuta a lo interno busca como finalidad las características y patrones relevantes o puntos de interés, que serían las zonas más negras y las zonas más blancas, luego cada característica es comparada y contrastada con las previas, descartando así la mayoría de puntos que son menos relevantes, mejorando así cada iteración o etapa de entrenamiento.

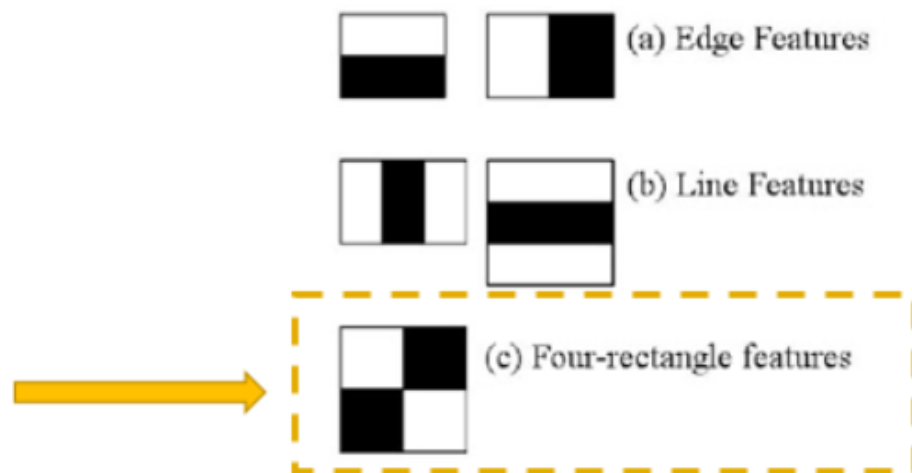


Figura 9 . Ejemplo feature buscado con Cascade Classifier

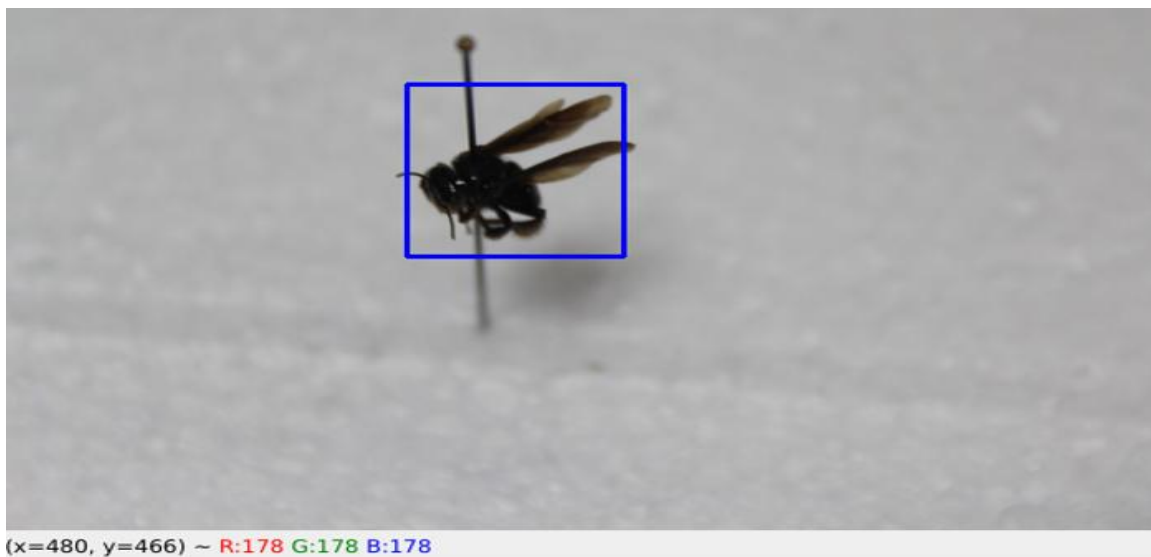
Fuente: Obtenido de OpenCV.org el 31 de marzo del 2021.

Dichos cascades posteriormente son agrupados en un *classifier*, el cual es un archivo en formato XML, el cual podrá ya ser incorporado dentro de las llamadas a nivel de código para analizar imágenes e identificar objetos. De acuerdo a la teoría, esto mejora la eficiencia,

Figura 10 . Ejemplo de identificación de abejas con Cascade Classifier

Fuente: Elaboración propia.

pasando por ejemplo de 16000 características a 6000, gracias al descarte que se logra durante el entrenamiento, centrándose en los puntos de interés verdaderamente relevantes. Siguiendo también la teoría, se dice que al menos 200 características identificadas pueden llegar a dar un 95% de eficacia en la identificación final. (*OpenCV*, 2021). Como podemos observar, el nombre de Cascade Classifier, hace honor al proceso que sigue para poder realizar el entrenamiento y comparar imágenes de entrada a manera de cascada, es decir, por fases. Si una imagen irrumpe en alguna fase, se desecha y si cumple con cada fase, podríamos encontrarnos en presencia del objeto a identificar. Por ejemplo, en vez de comparar 6000 características sobre una imagen, se agrupan dichas características por etapas, si una imagen acierta todas las etapas, estamos en presencia del objeto buscado (*OpenCV*, 2021).



Con las tecnologías y elementos de la arquitectura del sistema explicados, a continuación, se describe con mayor detalle el proceso de desarrollo e integración que se propone:

Django se propone del lado del *backend* del sistema, permitiendo tener en ejecución un servidor preparado para procesar cualquier petición o solicitud que reciba.

Adicionalmente Django permite integrar distintos gestores de base de datos, por lo que se propone utilizar MySQL al ser de código abierto y poseer un alto soporte dentro de la comunidad informática. Además, el modelo de base de datos que se utiliza es relacional, MySQL permite dicha implementación y un fácil manejo de consultas entre Django y dicho gestor. Por otro lado, se valora la particularidad que entre Django y MySQL se evita una muy común vulnerabilidad llamada inyección de SQL. Django permite el manejo de información a través de objetos parametrizados, lo que invalida este tipo de ataques dentro del sistema. Como parte del análisis y procesamiento de imágenes, se agregan la librería de OpenCV y la creación de todas las funciones auxiliares necesarias para hacer uso de dicha librería, analizar, procesar y obtener los resultados deseados con respecto a las imágenes a caracterizar.

Por el lado del *frontend*, se propone utilizar Ionic como *framework* de desarrollo, ya que con base en el mismo código podemos generar tanto la aplicación móvil como el sistema gestor. Adicionalmente, y gracias al sistema de acceso por tipo de usuario, se limita a un usuario de tipo regular el poder realizar operaciones de mantenimiento dentro del sistema, a diferencia de un administrador que si tiene dichos privilegios. Cabe mencionar que para el frontend se utilizan tanto tags a nivel de HTML5 como también tags propietarios y personalizados por Ionic. Además, se utilizará SASS como acelerador de estilos con base en codificación CSS y finalmente la lógica del cliente se programará bajo el *framework* Angular.

Para permitir la comunicación entre Django y un cliente, se realiza la implementación de un RESTful API, el cual permite las distintas consultas y peticiones hacia el servidor desde un cliente por medio de los métodos GET, PUT, POST y DELETE.

Con respecto a las pruebas, se realizan 2 tipos diferentes:

1. Pruebas funcionales: En esta fase, se prueban cada una de las funciones programadas tanto en el backend como en el frontend, garantizando que

realizan la funcionalidad para la cual fueron creadas. La meta de esta fase es depurar lo más posible el código de manera que cuando se encuentren problemas, se analicen, se depure el código necesario, se solvete el problema y se pruebe de nuevo para verificar que fue corregido.

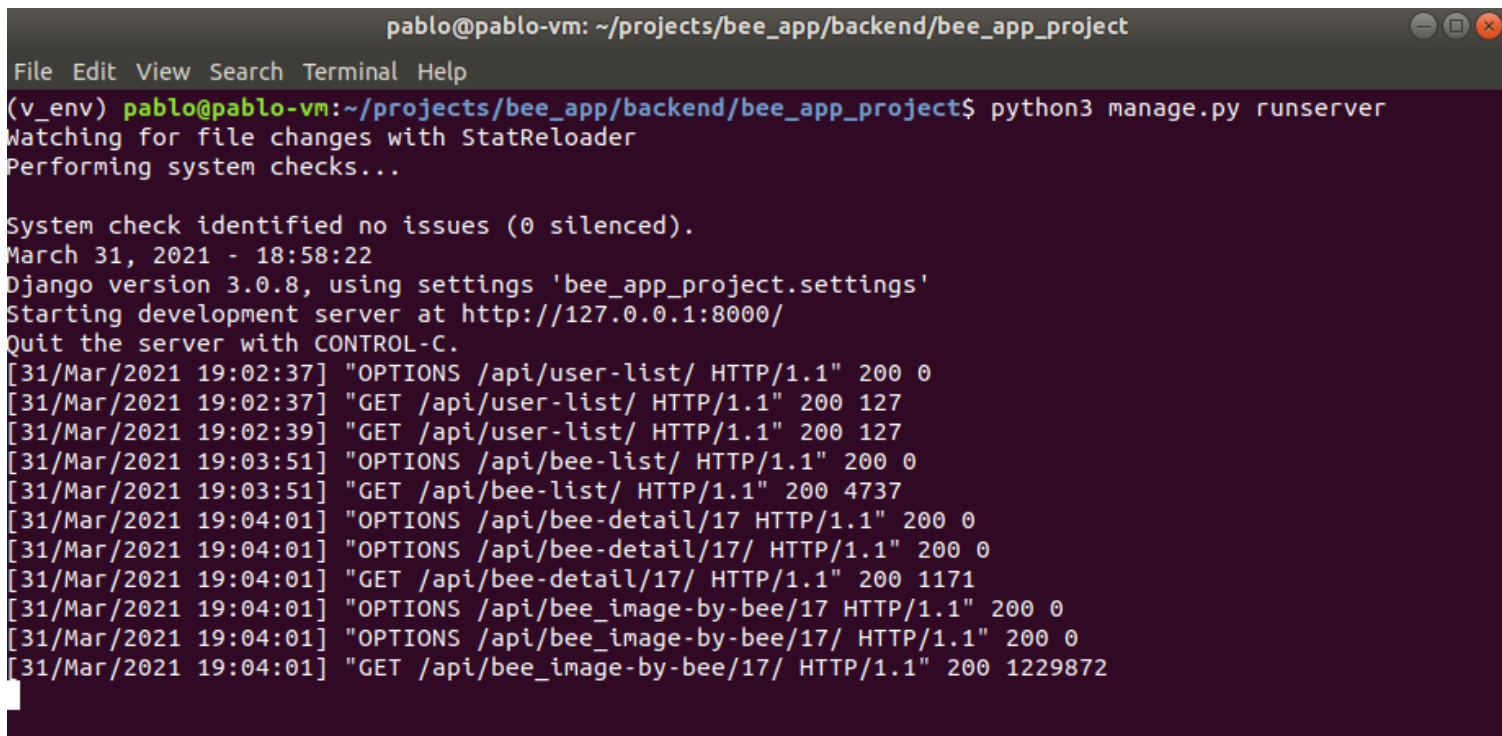
2. Pruebas a nivel de sistema e integración: En esta fase, se prueba el sistema como un todo, por lo que se procede a comprobar que el servidor responde a las múltiples solicitudes del cliente y que el cliente muestra la información de manera correcta, realizando las validaciones pertinentes. Por otro lado, una vez que el sistema esté correctamente depurado y en funcionamiento, se abarcan las pruebas relacionadas con la identificación de abejas utilizando las imágenes enviadas desde los clientes al servidor, para comprobar los tiempos de respuesta de la aplicación y la precisión en la identificación de imágenes, con base en una lista de posibilidades de especies ante la cual se podría estar.

Una vez que las pruebas fueron finalizadas, se recopilan los resultados, se analizan y se corrobora que la solución propuesta cumple con las expectativas y objetivos de la investigación. Finalmente, a partir del análisis de resultados se determinan las conclusiones, limitaciones y recomendaciones, las cuales se presentan en los capítulos posteriores de este documento.

2. Validación de la propuesta

Servidor basado en Django

Una vez que el servidor está en ejecución, escucha peticiones del cliente por medio del RESTful API implementado y consecuentemente envía una respuesta. Cabe mencionar que el servidor basado en Django es independiente del sistema operativo y puede instalarse ya sea en Windows o Linux, sin requerimientos específicos a nivel de hardware, ya que esto quedaría sujeto a los recursos disponibles por parte del CINAT o la escuela de Informática de la Universidad Nacional, en este caso.



```
pablo@pablo-vm: ~/projects/bee_app/backend/bee_app_project
File Edit View Search Terminal Help
(v_env) pablo@pablo-vm:~/projects/bee_app/backend/bee_app_project$ python3 manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
March 31, 2021 - 18:58:22
Django version 3.0.8, using settings 'bee_app_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
[31/Mar/2021 19:02:37] "OPTIONS /api/user-list/ HTTP/1.1" 200 0
[31/Mar/2021 19:02:37] "GET /api/user-list/ HTTP/1.1" 200 127
[31/Mar/2021 19:02:39] "GET /api/user-list/ HTTP/1.1" 200 127
[31/Mar/2021 19:03:51] "OPTIONS /api/bee-list/ HTTP/1.1" 200 0
[31/Mar/2021 19:03:51] "GET /api/bee-list/ HTTP/1.1" 200 4737
[31/Mar/2021 19:04:01] "OPTIONS /api/bee-detail/17 HTTP/1.1" 200 0
[31/Mar/2021 19:04:01] "OPTIONS /api/bee-detail/17/ HTTP/1.1" 200 0
[31/Mar/2021 19:04:01] "GET /api/bee-detail/17/ HTTP/1.1" 200 1171
[31/Mar/2021 19:04:01] "OPTIONS /api/bee_image-by-bee/17 HTTP/1.1" 200 0
[31/Mar/2021 19:04:01] "OPTIONS /api/bee_image-by-bee/17/ HTTP/1.1" 200 0
[31/Mar/2021 19:04:01] "GET /api/bee_image-by-bee/17/ HTTP/1.1" 200 1229872
```

Figura 11 . Servidor basado en Django en ejecución

Fuente: Elaboración propia.

RESTful API

A continuación, se detallan las diferentes peticiones disponibles a través del API implementado:

Api Overview

OPTIONS GET

GET /api/

```
HTTP 200 OK
Allow: GET, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "List Bee": "/bee-list/",
  "List User": "/user-list/",
  "List Bee Image": "/bee_image-list/",
  "List Temp Image": "/temp_image-list/",
  "List Bee Image by Bee ID": "/bee_image-by-bee/<str:bee_pk>",
  "Detail View Bee": "/bee-detail/<str:pk>",
  "Detail View User": "/user-detail/<str:pk>",
  "Detail View Bee Image": "/bee_image-detail/<str:pk>",
  "Detail View Temp Image": "/temp_image-detail/<str:pk>",
  "Create Bee": "/bee-create/",
  "Create User": "/user-create/",
  "Create Bee Image": "/bee_image-create/",
  "Create Temp Image": "/temp_image-create/",
  "Update Bee": "/bee-update/<str:pk>/",
  "Update User": "/user-update/<str:pk>/",
  "Update Bee Image": "/bee_image-update/<str:pk>",
  "Update Temp Image": "/temp_image-update/<str:pk>",
  "Delete Bee": "/bee-delete/<str:pk>",
  "Delete User": "/user-delete/<str:pk>",
  "Delete Bee Image": "/bee_image-delete/<str:pk>",
  "Delete Temp Image": "/temp_image-delete/<str:pk>",
  "Generate Imgs": "/generate_imgs/"
}
```

Figura 12 . Índice del API creado

Fuente: Elaboración propia.

Descripción de las peticiones:

Petición	Descripción
List Bee	Lista todas las abejas almacenadas de la base de datos
List User	Lista todos los usuarios almacenados de la base de datos.
List Bee Image	Lista todas las imágenes (en formato base64) almacenadas en la base de datos.
List Temp Image	Lista la imagen temporal (en formato base64) almacenada en la base de datos.
List Bee Image by Bee ID	Lista una imagen por ID almacenada en la base de datos.
Detail View Bee	Lista una abeja en detalle por ID almacenada en la base de datos.
Detail View User	Lista un usuario en detalle por ID almacenado en la base de datos.
Detail View Bee Image	Lista una imagen asociada a una abeja en detalle, de acuerdo con su ID almacenada en la base de datos.

Detail View Temp Image	Lista una imagen temporal de una abeja en detalle, de acuerdo con su ID almacenada en la base de datos.
Create Bee	Crea una abeja con base en sus atributos y valores dados.
Create User	Crea un usuario con base en sus atributos y valores dados.
Create Bee Image	Crea una imagen asociada a una abeja, con base a sus atributos y valores dados.
Create Temp Image	Crea una imagen asociada a una abeja, con base a sus atributos y valores dados.
Update Bee	Actualiza los valores de los atributos de una abeja en específico.
Update User	Actualiza los valores de los atributos de un usuario en específico.
Update Bee Image	Actualiza los valores de los atributos de una imagen asociada a una abeja en específico.
Update Temp Image	Actualiza los valores de los atributos de una imagen temporal.
Delete Bee	Elimina una abeja en específico.
Delete User	Elimina un usuario en específico.
Delete Bee Image	Elimina una imagen en específico asociada a una abeja.
Delete Temp Image	Elimina la imagen temporal creada.
Generate Imgs	Genera todas las imágenes en formato base64 actuales en la base de datos a PNG, ordenadas en diferentes folders, dentro de los assets del backend, por especie.

Tabla 2. Descripción de peticiones

Cliente basado en Ionic

Una vez que el servidor esté esperando peticiones y el API listo para ser utilizado, se procede a ejecutar el cliente, en este caso el sistema gestor o como aplicación móvil:

Vista web

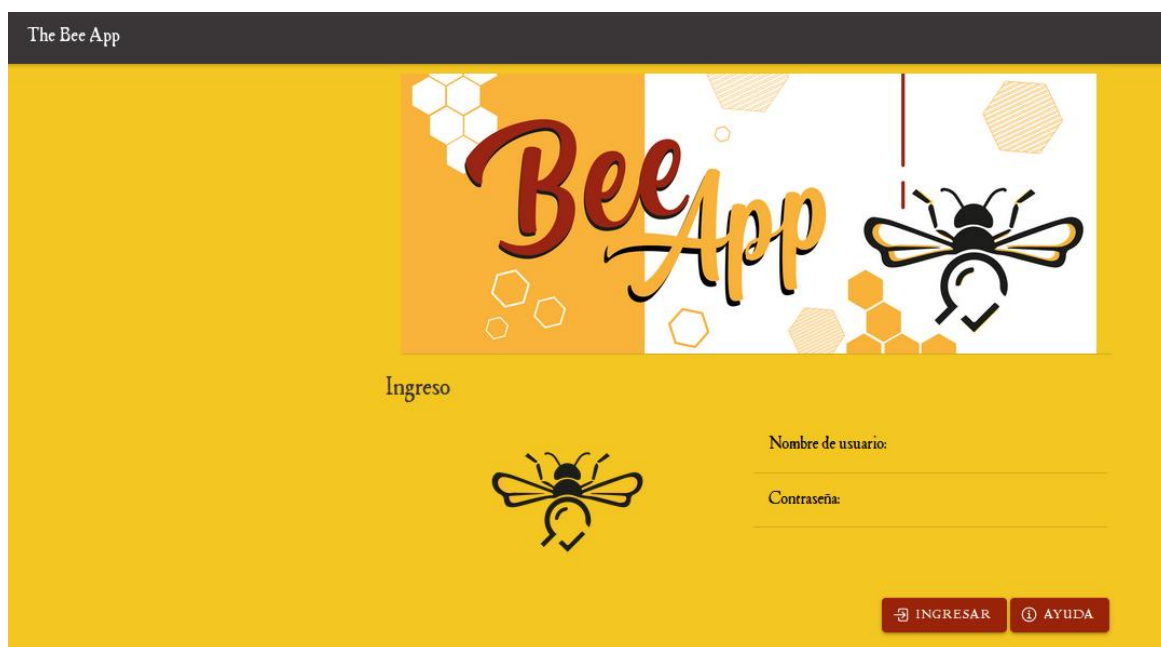


Figura 13 . Login vista web

Fuente: Elaboración propia.

Vista móvil



Figura 14 . Login vista móvil

Fuente: Elaboración propia.

Al ingresar al sistema, si se cuentan con derechos de administrador, se pueden observar varias opciones, entre ellas dos relacionadas a la mantenibilidad del sistema o como bien se ha descrito, sistema gestor. También, el administrador cuenta con la opción de realizar capturas y enviarlas a análisis hacia el servidor:



Figura 15 . Menú principal vista web

Fuente: Elaboración propia.

Dentro de cada opción de ABEJAS o USUARIOS, existen las diferentes opciones de mantenibilidad como lo son: agregar, eliminar, listar y actualizar:



Figura 16 . Menú de usuarios vista web

Fuente: Elaboración propia.



Figura 17 . Menú de abejas vista web

Fuente: Elaboración propia.

Cabe destacar que también se cuentan con las mismas opciones de mantenimiento para un usuario de tipo administrador. La diferencia es que, en el Menú de Abejas, existe una opción llamada GENERAR IMÁGENES, la cual permite, con base en todas las imágenes existentes en la base de datos en formato base64, ser generadas en formato PNG y categorizarlas por especie en diferentes folders. El resultado se puede visualizar en la carpeta de utils/assets del lado del servidor, tal como se muestra en la figura 18.

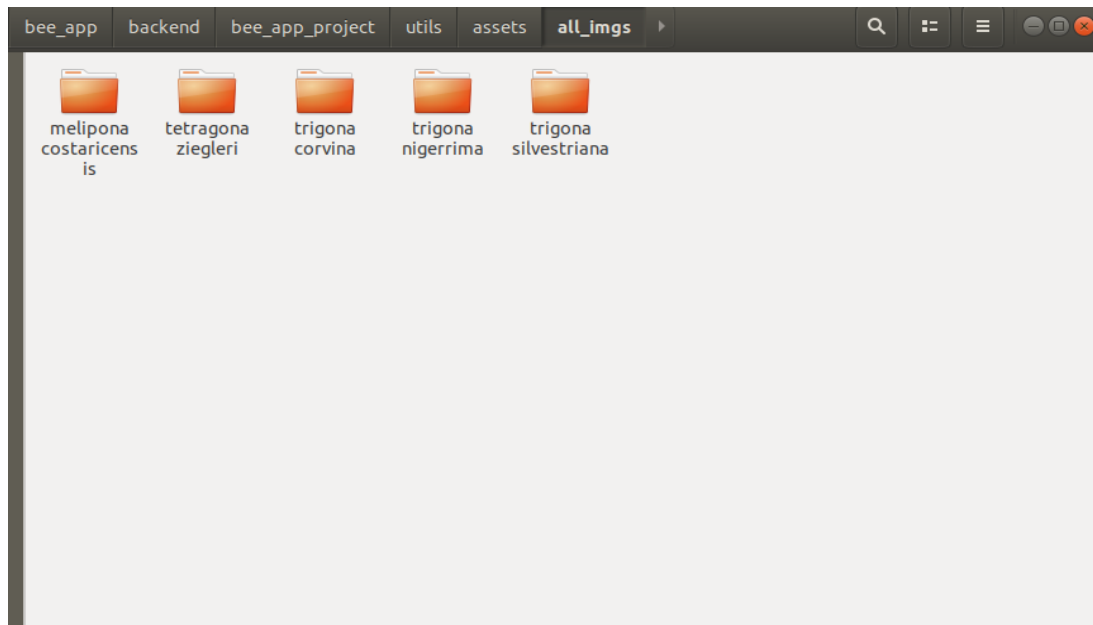


Figura 18 . Generación de imágenes por folder

Fuente: Elaboración propia

Como se mencionó anteriormente, si el usuario es de tipo administrador, tendrá todos los privilegios de acceso, mientras que si se ingresa como un usuario tipo regular, el cual está previamente registrado en el sistema, las opciones serán reducidas a solamente realizar capturas y consultar la galería de abejas.

Tipo de usuario	Privilegios
administrador	Consultar galería, Analizar imágenes, Menú de Usuarios, Agregar usuarios, Eliminar usuarios, Consultar usuarios, Actualizar usuarios, Menú de abejas, Agregar abejas, Eliminar abejas, Consultar abejas, Actualizar abejas, Generar imágenes.
regular	Consultar galería, Analizar imágenes.

Tabla 3. Tipo de usuarios y privilegios

Fuente: Elaboración propia



Figura 19 . Menú principal vista móvil regular

Fuente: Elaboración propia

Como bien se menciona, la funcionalidad principal es la de poder realizar capturas y enviar a análisis, con la finalidad de obtener una respuesta acerca de ante cuál especie de abeja nos encontramos, para esto se selecciona la opción CAPTURAR.

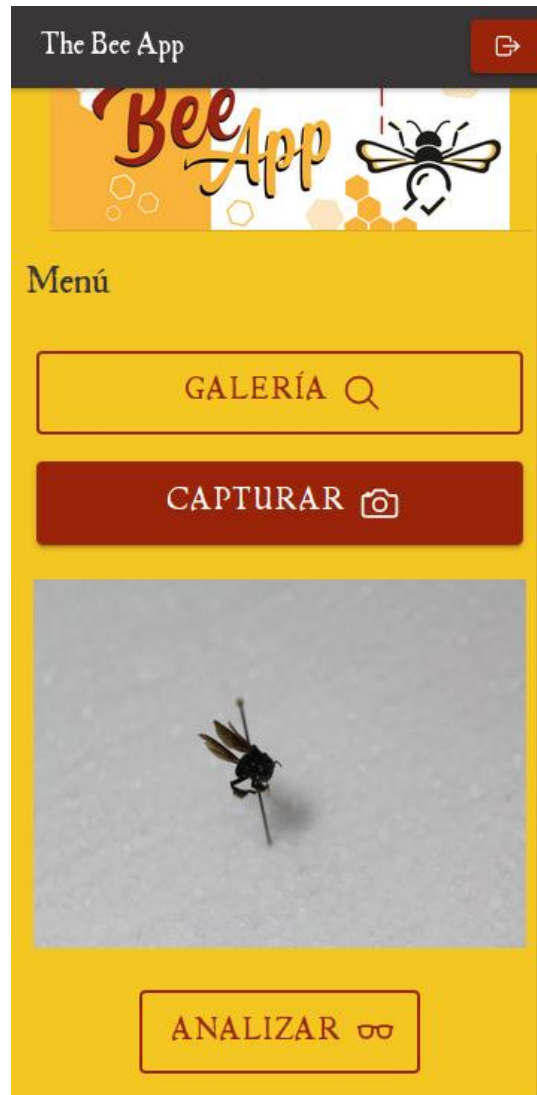


Figura 20 . Captura vista móvil

Fuente: Elaboración propia.

Una vez realizada una captura se habilita la opción de ANALIZAR, lo que enviará dicha imagen al servidor para ser analizada y posteriormente obtener una lista de abejas dentro de las cuales podría pertenecer en cuanto a su especie. En el ejemplo se utiliza una especie *Trigona Silvestriana*, como se muestra en la figura 20.

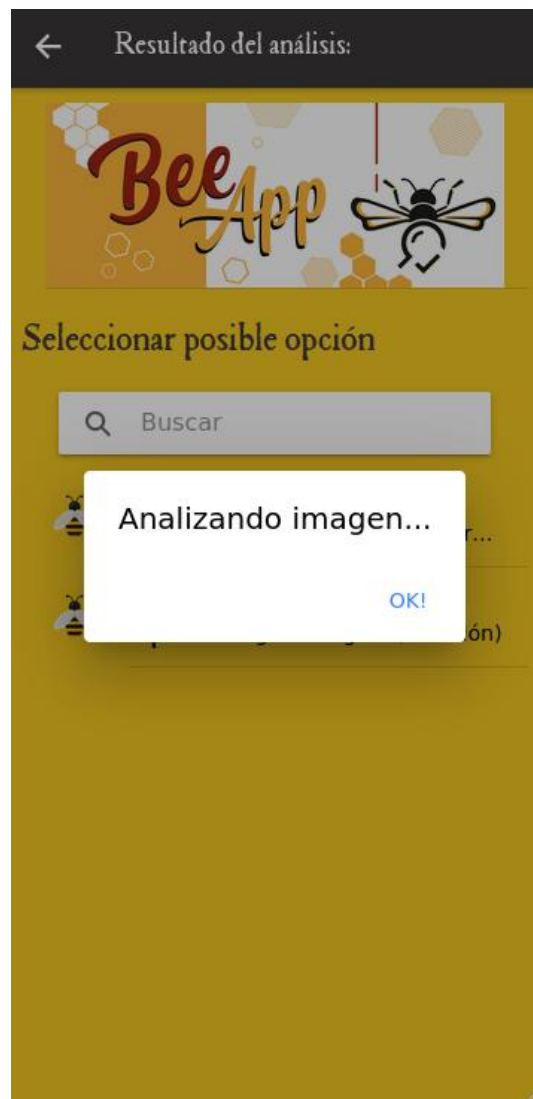


Figura 21 . Mensaje analizando imagen

Fuente: Elaboración propia.

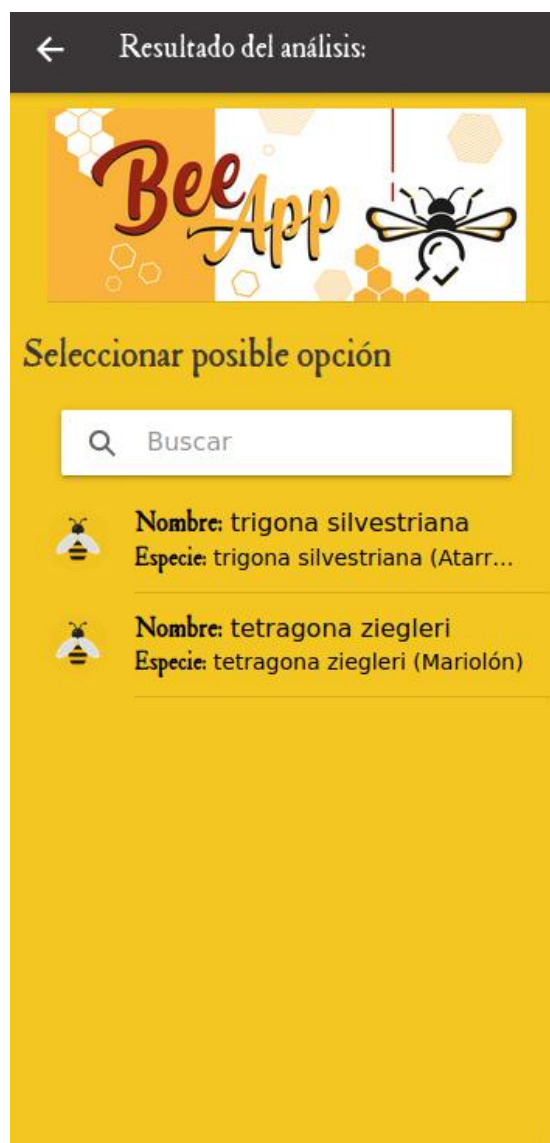


Figura 22 . Resultado de análisis vista móvil

Fuente: Elaboración propia.

Con la lista resultante, se puede seleccionar la opción para visualizar la información referente a dicha abeja.



Figura 23 . Ejemplo ficha técnica

Fuente: Elaboración propia.

Por otro lado, existe actualmente la opción de consultar la galería directamente desde el menú principal y listar de esta manera las abejas que se encuentran en la base de datos del sistema. Podemos inclusive aplicar filtros de búsqueda por nombre o especie.

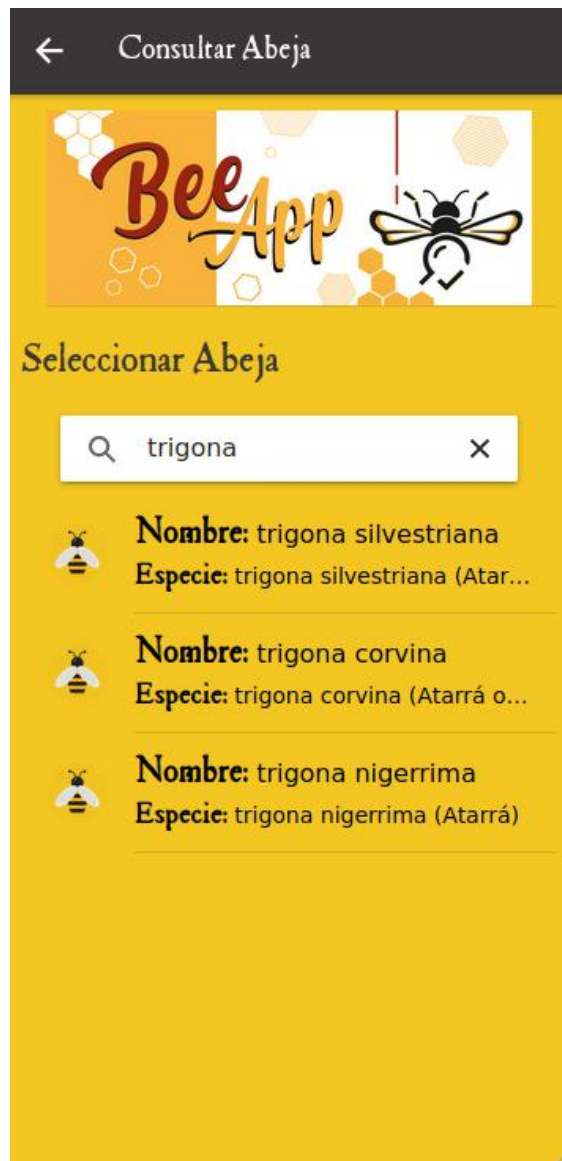


Figura 24 . Consultar abeja vista móvil

Fuente: Elaboración propia.

Funciones relevantes

Se detallan algunas de las funciones más relevantes a nivel de código dentro del sistema, ya que cumplen un papel muy importante dentro de toda la manipulación y el procesamiento de imágenes.

```

public async capturePhoto() {

    // Re-Initialice bee photo capture
    this.capture = new TempImageModel();

    // Take a photo as base64 resulted
    const capturedPhoto = await Camera.getPhoto({
        resultType: CameraResultType.DataUrl,
        source: CameraSource.Camera,
        quality: 100
    });
    // Create model based on photo captured
    this.capture.url = capturedPhoto.dataUrl;
}

public sendToAnalyze(){
    this.alertController.create({
        header: 'Analizando imagen...',
        buttons: ['OK!']
    }).then(alertController =>{
        alertController.present();
    });
}

```

Figura 25 . Función para capturar imagen

Fuente: Elaboración propia.

Tal y como se muestra en la figura 25, la función es utilizada con la finalidad de capturar una imagen por medio de la cámara del dispositivo móvil. De esta manera, con base en dicha imagen se crea un objeto de tipo imagen temporal, que será enviada en formato base64 hacia el servidor para su debido procesamiento. Cabe mencionar que el objeto creado de tipo imagen temporal tiene como parte de sus atributos un ID autogenerado y además un URL, el cual alberga el valor de la imagen en base64 que para fines prácticos se visualiza como un *long char* o *string*. Adicionalmente, se muestra un mensaje al usuario final una vez que la imagen fue recibida y está lista para su procesamiento.

```
# convert base64 image in the database to png image
def base64_to_png(base64Image, output_name):
    im = Image.open(BytesIO(base64.b64decode(base64Image.split(',')[1])))
    im.save(output_name, 'PNG')
```

Figura 26 . Función para convertir a png

Fuente: Elaboración propia.

Como se mencionó anteriormente, se utilizan imágenes tanto en formato PNG como en base64 para su debido procesamiento, debido a su facilidad y practicidad de poder ser guardado como un string en la base de datos. La función en la figura 26, permite realizar dicha conversión, en este caso convirtiendo imágenes dentro de la base de datos, para realizar la conversión a PNG.

```
def tempImageCreate(request):
    match_result = []
    serializer = TempImageSerializer(data=request.data)
    if serializer.is_valid():
        base64_to_png(serializer.data['url'], TEMP_IMG + 'test_img.png')
        match_result = cascade_classifier.classify(TEMP_IMG + 'test_img.png')
        result_serializer = get_bees_match(match_result)
        if len(match_result) > 0:
            return Response(result_serializer.data)
        else:
            return Response(False)
    return Response(serializer.errors)
```

Figura 27 . Función crear imagen temporal y conversiones

Fuente: Elaboración propia.

Seguidamente, de acuerdo con la figura anterior, se utilizan las funciones auxiliares tanto del lado de OpenCV como del backend, con el propósito de realizar una conversión a la inversa de base64 a PNG, para posteriormente procesar dicha imagen en conjunto con los clasificadores previamente entrenados y, una vez que se obtiene la lista de aserción por clasificador, se busca dentro de la base de datos todas las referencias con base en los clasificadores y las especies que especifican cada uno. Finalmente, basta con responder si

la lista es mayor a uno, lo que quiere decir que al menos una especie fue identificada. En caso contrario, no habría aserción con respecto a la imagen recibida.

```
CLASSIFIERS = 'utils/open_cv/cascade_classifier/classifiers/classifiers.json'
HIT_LIST = []

def classify(entry_image):
    HIT_LIST = []
    with open(CLASSIFIERS) as my_classifiers:
        data = json.load(my_classifiers)
        for classifier in data:
            print(classifier)
            print(data[classifier])
            bee_classifier = cv2.CascadeClassifier(data[classifier])
```

Figura 28 . Función de clasificación y lista de aserción

Fuente: Elaboración propia.

Para poder realizar un efectivo manejo de aserción por especie con respecto a una imagen de entrada, se implementa un archivo .json con la finalidad de compilar cada clasificador previamente entrenado, considerando la especie que permite identificar. De esta manera, se compara la imagen procesada contra cada uno de los clasificadores existentes en el sistema y si se evidencia una aserción por parte de la imagen con el clasificador, se puede especificar qué especie es, devolviendo así como resultado final toda la lista de aserciones.

```
img = cv2.imread(entry_image)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Figura 29 . Procesamiento de imágenes

Fuente: Elaboración propia.

De acuerdo con la figura 29, una imagen recibida se procesa de manera tal que se convierte primero a escala de grises, realizando su debido proceso de umbralización. Luego se procede a detectar los puntos de interés contra un clasificador cargado dentro de la iteración de cada uno de los clasificadores entrenados.

```
HIT_LIST.append(classifier)
for (q, w, e, r) in bees:
    img=cv2.rectangle(img, (q,w), (q+e, w+r), (255, 0, 0), 2)
cv2.imshow('img', img)
```

Figura 30 . Identificación y cuadriculación de puntos de interés

Fuente: Elaboración propia.

Cada aserción es guardada en una lista para ser verificada contra la base de datos actual del sistema y adicionalmente se usan funciones de marcado provistas por openCV Se dibuja sobre la imagen resultante la zona dentro de la imagen en la cual hubo aserción, la cual se espera que sea la representación de la abeja.

Resultados

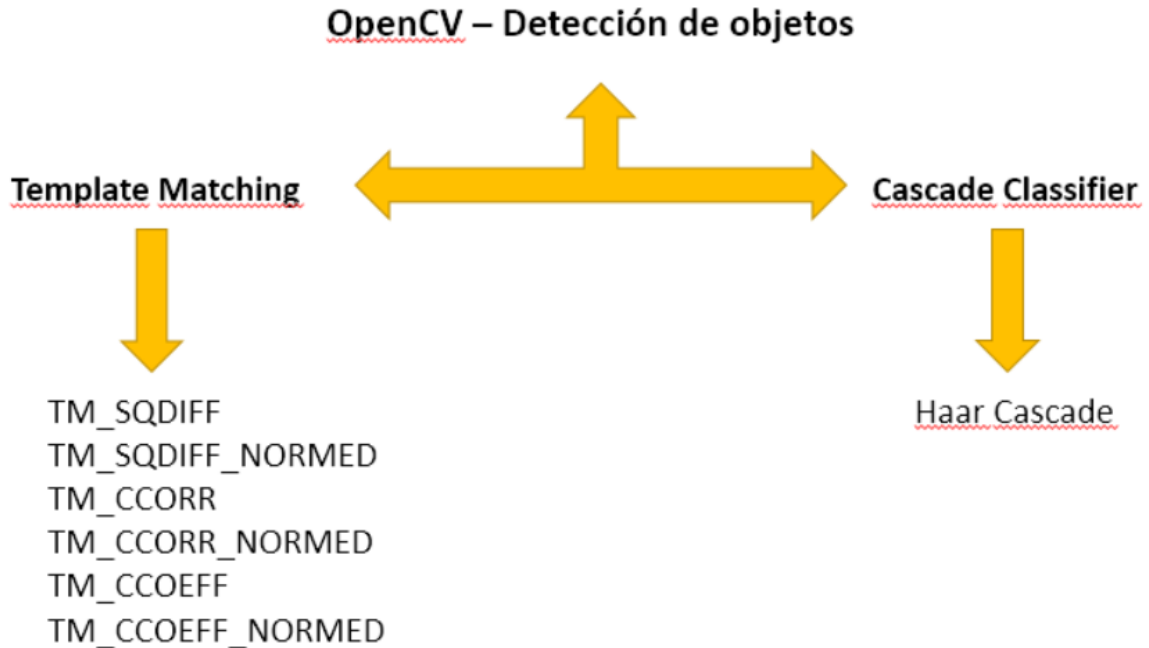


Figura 31 . Diagrama detección de objetos.

Fuente: Elaboración propia.

Como se ha mencionado con anterioridad, la finalidad buscada es la detección de objetos, en este caso en específico, especies de abejas *Apidae Meliponini*, por lo que, con base en el esquema anterior, se procede a presentar los resultados obtenidos tanto de la vertiente Template Matching, con sus 6 diferentes métodos, como de la vertiente Cascade Classifier con el proceso Haar Cascade.

Resultados – Template Matching

El proceso seguido para recolectar resultados fue el siguiente, utilizando como objeto de prueba una abeja *melipona costaricensis*:

- 1- Contar con la imagen base, cargarla en una variable dentro del código para posteriormente ser captada y procesada por medio de OpenCV:



Figura 32 . Imagen base ejemplo 1

Fuente: Obtenido de miel dorada.net el 31 de marzo del 2021.

- 2- Recibir la imagen de entrada, cargarla en una variable dentro del código para posteriormente ser captada y procesada por medio de OpenCV:



Figura 33 . Imagen de entrada ejemplo 1

Fuente: Obtenido de mieldorada.net el 31 de marzo del 2021.

- 3- Obtener el resultado mediante la aplicación de cada uno de los métodos ofrecidos por *Template Matching* para el proceso de detección de la imagen de entrada con respecto a la base por medio de OpenCV:

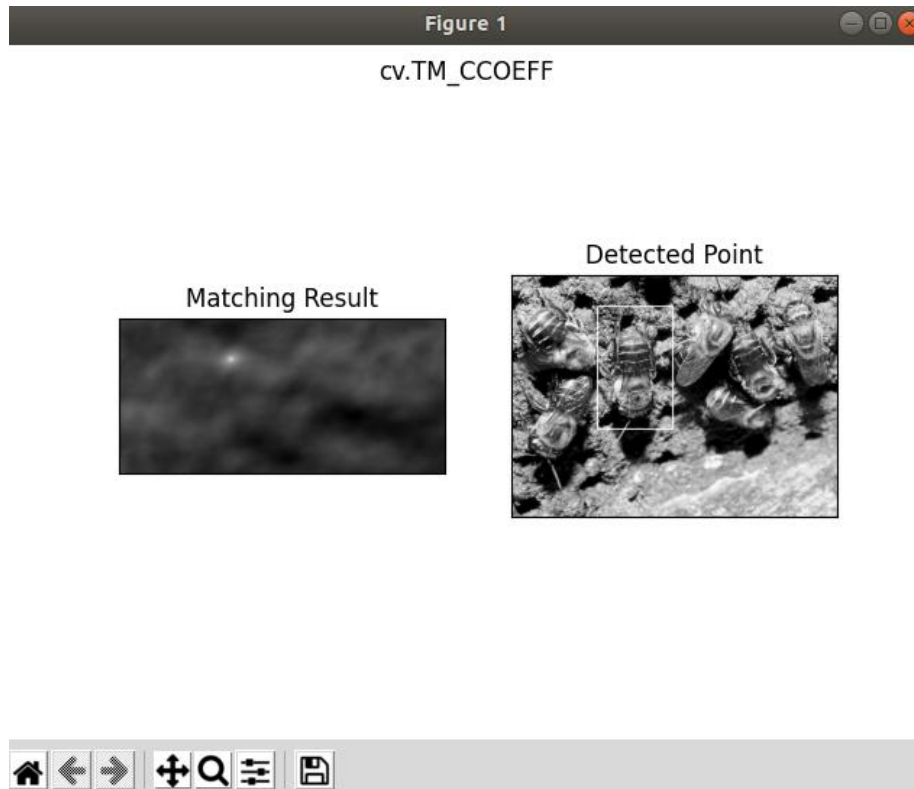


Figura 34 . Análisis y captura template matching 1

Fuente: Elaboración propia & obtenido de mieldorada.net el 31 de marzo del 2021.

- 4- Repetir dicho proceso utilizando cada método disponible por Template Matching y tabular la información:

Esperado: Aserción

TM_CCOEFF	Acertó
TM_CCOEFF_NORMED	Acertó
TM_CCORR	No acertó
TM_CCORR_NORMED	Acertó
TM_SQDIFF	Acertó
TM_SQDIFF_NORMED	Acertó

Figura 35 . Resultado Template Matching ejemplo 1

Fuente: Elaboración propia

Este proceso se repitió en total 128 veces en total, abarcando diferentes escenarios y especies, con la finalidad de observar, analizar y deducir la efectividad de cada método tanto a la hora de identificar una especie, como de no identificarla, lo que se traduce a la capacidad de discernir, o discriminar la especie dentro de un grupo.

Otros ejemplos:

Base



Figura 36 . Base 1 trigona silvestriana

Fuente: Obtenido de ecosdelbosque.com el 31 de marzo del 2021.

Entradas y resultados

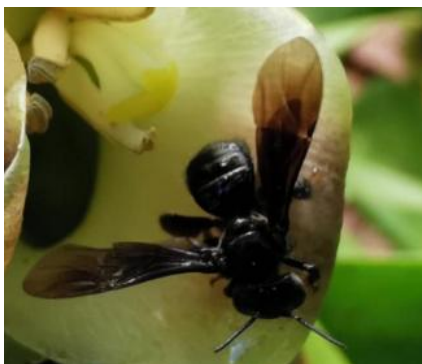


Figura 37 . Entrada 2 trigona silvestriana

Fuente: Obtenido de ecosdelbosque.com el 31 de marzo del 2021

Esperado: Aserción

TM_CCOEFF	Acertó
TM_CCOEFF_NORMED	Acertó
TM_CCORR	Acertó
TM_CCORR_NORMED	Acertó
TM_SQDIFF	Acertó
TM_SQDIFF_NORMED	Acertó

Figura 38 . Resultados trigona silvestriana 2

Fuente: Elaboración propia



Figura 39 . Entrada trigona silvestriana 3

Fuente: Obtenido de

Colombia.inaturalist.org el 31 de marzo

Esperado: Aserción

TM_CCOEFF	Acertó
TM_CCOEFF_NORMED	Acertó
TM_CCORR	Acertó
TM_CCORR_NORMED	Acertó
TM_SQDIFF	Acertó
TM_SQDIFF_NORMED	Acertó

Figura 40 Resultado Trigona Silvestriana 3

Fuente: Elaboración propia

Para cada fase de pruebas se recolectó la información y se tabuló de una manera en que se puedan analizar y comparar los resultados entre cada método para poder medir cuál podría ser la mejor opción.

	Prueba 1	Prueba 2	Prueba 3	Prueba 4	Prueba 5	Prueba 6	Porcentaje de éxito	Porcentaje de fallo
TM_CCOEFF							49,98	50,2
TM_CCOEFF_NORMED							49,98	50,2
TM_CCORR							33,32	66,68
TM_CCORR_NORMED							49,98	50,2
TM_SQDIFF							49,98	50,2
TM_SQDIFF_NORMED							33,32	66,68

Tabla 4 . Resumen de pruebas template matching

Fuente: Elaboración propia

Porcentaje de éxito	Método
100%	
90%	
80%	
70%	
60%	
50%	TM_CCOEFF TM_CCOEFF_NORMED TM_CCORR_NORMED TM_SQDIFF
40%	
30%	TM_CCORR TM_CCORR
20%	
10%	
0%	

Tabla 5 . Resumen de metodos template matching

Fuente: Elaboración propia.

Con la última tabla, se midió en cada fase de pruebas cuál método es mejor, se promediaron todos los resultados y se obtuvieron los porcentajes correspondientes.

Resultados - Cascade Classifier

El proceso en este caso fue tabular todos los datos en cada fase de pruebas para procesar de una manera similar al caso con Template Matching, el cálculo, promedio y conclusiones de los resultados obtenidos.

Haar cascade - Segunda fase de pruebas							
	Notas:	Acerción	El porcentaje de éxito por clasificador, describe la capacidad de identificar verdaderamente ante cual especie de abeja nos				
		Sin acerción	El porcentaje de éxito por abeja, describe la capacidad de descarte o discriminación de la especie probada dentro de un grupo o ámbito de varias especies de abejas dadas				
Nomenclatura de especies	Entrada/Prueba	Prueba 1	Prueba 2	Prueba 3	Prueba 4	Prueba 5	Porcentaje de éxito obtenido por abeja
		melipona_costaricensisClassifier	tetragona_ziegleriClassifier	trigona_corvinaClassifier	trigona_nigerrimaClassifier	trigona_silvestrianaClassifier	
tc=trigona corvina	mc17	Esperado: Acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	60%
tn=trigona nigerrima	mc27	Esperado: Acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	80%
ts= trigona silvestriana	mc32	Esperado: Acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	40%
mc=melipona costaricensis	tz12	Esperado: Sin acerción	Esperado: Acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	40%
tz=tetragona ziegleri	tz26	Esperado: Sin acerción	Esperado: Acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	80%
	tz32	Esperado: Sin acerción	Esperado: Acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	80%
	tc16	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Acerción	Esperado: Sin acerción	Esperado: Sin acerción	60%
	tc19	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Acerción	Esperado: Sin acerción	Esperado: Sin acerción	60%
	tc35	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Acerción	Esperado: Sin acerción	Esperado: Sin acerción	80%
	tn17	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Acerción	Esperado: Sin acerción	60%
	tn29	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Acerción	Esperado: Sin acerción	60%
	tn34	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Acerción	Esperado: Sin acerción	60%
	ts25	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Acerción	60%
	ts29	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Acerción	80%
	ts33	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Sin acerción	Esperado: Acerción	80%
	Porcentaje de éxito obtenido por clasificador	93,33%	53,33%	60%	66,67%	53,33%	Promedio: (Por fila/columna) 65%/65%
						Total de pruebas realizadas:	75

Tabla 6 . Resumen resultados Cascade Classifier

Fuente: Elaboración propia.

Para comprender la tabla de resultados anterior, se comienza por entender la nomenclatura utilizada para cada uno de los objetos de prueba situados en la columna Entrada/Especie. Posteriormente, cada prueba será una iteración distinta, en donde la variante será el clasificador utilizado, ya que cada uno representa una especie. Dentro de cada iteración de pruebas, los mismos objetos de pruebas fueron sometidos para ser procesados con cada clasificador, por lo que podemos ahora ubicarnos en los porcentajes de éxito citados tanto en la última columna como en la última fila, esto quiere decir que una misma imagen, relacionada a una especie, fue sometida a prueba contra todos los clasificadores, con el fin de obtener la capacidad de discriminación o descarte, ya que el propósito es que solo el clasificador afin identifique la entrada correspondiente. Por otro lado, como todas las entradas fueron utilizadas a su vez contra el mismo clasificador, esto denota la capacidad de identificar ante cuál especie de abeja nos encontramos. Ahora, pasamos a las definiciones de aserción y sin aserción. Una aserción significa que la

operación de identificación sí tuvo un hallazgo, mientras que sin aserción, es lo opuesto, no hubo un hallazgo, esto con respecto a la entrada y los clasificadores previamente entrenados y cargados en el sistema. Además, se indica cuál era el esperado en cada uno de las pruebas. Con las letras en blanco se distinguen las que se esperaban que fueran acertadas ya que se probaron contra su respectivo clasificador.

Finalmente, pasamos a los datos presentados por los porcentajes de éxito, como las leyendas en las notas lo definen:

- El porcentaje de éxito por clasificador, describe la capacidad de identificar verdaderamente ante cuál especie de abeja nos encontramos.
- El porcentaje de éxito por abeja, describe la capacidad de descarte o discriminación de la especie probada dentro de un grupo o ámbito de varias especies de abejas.

Y a su vez podemos también se promediaron ambos resultados como se indica en la última fila y columna de la tabla.

Resultados – Resumen

Se realizaron un total de 238 pruebas entre Template Matching y Cascade Classifier denotando los siguientes puntos:

Cascade Classifier

- Hubo una aserción entre un 60% y 80% a nivel de identificación de la especie de la abeja dentro de un grupo.
- Hubo un descarte o discriminación dentro de un 53% y 93% de la especie dentro de un grupo de abejas.
- Presenta un menor margen de error gracias a que puede mejorarse con el entrenamiento adecuado. Eso se evidencia con las mejoras que hubo entre cada fase de pruebas y los recursos a nivel de banco de imágenes utilizado.
- Caracteriza una mayor complejidad en cuanto a uso e implementación.
- Tiene un mayor uso de recursos solo durante el entrenamiento del algoritmo.

- Posee una mayor eficiencia y un menor uso de recursos una vez el algoritmo ya ha sido entrenado y es utilizado finalmente en conjunto con el sistema.
- Es más eficaz en comparación al Template Matching.

Template Matching

- Hubo una aserción entre un 30% y 80% a nivel de identificación de la especie de la abeja dentro de un grupo dado.
- Hubo un descarte o discriminación dentro de un 0% y 83% de la especie dentro de un grupo de abejas.
- No permite modificaciones u opciones mayores que los 6 métodos ya propiamente dados, por lo que su margen de error no puede ser mejorado mediante algún tipo de entrenamiento.
- Tiene menor complejidad en cuanto a uso e implementación.
- Posee mayor un uso de recursos del sistema y es menos eficaz en comparación al cascade classifier.

Mejor algoritmo a utilizar: Con base en los resultados anteriores, se concluye que el mejor algoritmo a utilizar es Cascade Classifier.

Con el fin de reforzar los datos obtenidos mediante el instrumento propio creado para Cascade Classifier y mostrado anteriormente en la tabla 4, también se sometió la información a una matriz de confusión, la cual es el estándar para medir la efectividad de algoritmos en machine learning. Se incluye esta matriz de confusión, específicamente una matriz binaria, con los resultados. Se destaca la precisión obtenida a través de la formula con base en su diagonal y también otros cálculos útiles disponibles:

		Actual		
Predicción		Positivo	Negativo	Total
	Positivo	8 (Verdaderos Positivo)	19 (Falso Positivos)	27
	Negativo	7 (Falsos Negativos)	41 (Verdaderos Negativos)	48
	Total	15	60	75

Medidas	Valor
Sensibilidad	53%
Especificidad	68%
Valor predictivo negativo	85%
Tasa de falsos positivos	31%
Tasa de descubrimiento de falsos	70%
Tasa de falsos negativos	46%
Precisión	65%

Tabla 7. Matriz de confusión Cascade Classifier

Fuente: Elaboración propia.

Una ventaja de contar con esta matriz de confusión para el algoritmo Cascade Classifier, es que permite evidenciar que su precisión es igual al porcentaje de eficacia mostrado con el instrumento propiamente creado en la tabla *Resumen resultados Cascade Classifier*, tanto para poder distinguir cual especie de abeja estamos analizando y también discriminar y descartar dicha especie dentro de un grupo de otras especies de abejas. Adicionalmente permite la medición del algoritmo con base en diferentes medidas y enfoques para posteriormente obtener nuevas conclusiones e incluso nuevos enfoques de investigación.

CAPÍTULO V

CONCLUSIONES Y RECOMENDACIONES

CAPÍTULO V: CONCLUSIONES Y RECOMENDACIONES

Conclusiones

A continuación, se presentan las principales conclusiones del proyecto realizado:

1. Las abejas no son animales fáciles de fotografiar ya que poseen características no visibles al ojo humano y que tampoco son perceptibles ante la cámara del dispositivo móvil, por ejemplo, poseen características singulares como franjas entre los ojos entre especies del mismo género o incluso entre géneros muy similares como *Partamona orizabaensis* y las *Trigonas*. También poseen características no visibles como rojizo en sus patas, mayor vello en el tórax o patas.
2. Existen también a nivel de captura de imágenes por medio de la cámara de los dispositivos móviles, problemas de enfoque, iluminación y limitaciones técnicas en cuanto a capacidades de la cámara y dispositivo móvil.
3. Por otro lado, hay características importantes para la clasificación de especies de abejas no visibles del todo, como por ejemplo el comportamiento de la abeja, social o asocial, también si son agresivas y territoriales, además de su zona donde comúnmente es hallada y también características tanto de la miel como del polen que producen en los panales.
4. Con base en las entrevistas realizadas, se denota que la caracterización de abejas difiere según la experiencia entre los taxónomos, ya que un experto puede dar un veredicto de acuerdo con sus observaciones, pero siguiendo distintos procesos de identificación, como el de llaves que maneja actualmente el CINAT o por las entradas de su piquera.
5. Se propone utilizar Cascade Classifier como algoritmo de caracterización de especies de abejas en conjunto con el sistema y aplicación móvil implementada.
6. Adicionalmente, con base en las pruebas realizadas, se notan mejores resultados utilizando una mayor cantidad de imágenes en un ambiente controlado, ya que variables como la posición de las fotografías o imágenes, el ángulo de captación, el ruido provocado por el ambiente, ya sea natural como el fondo, la iluminación, el

acercamiento, o en el laboratorio como el fondo utilizado, los alfileres que sostienen las muestras, las sombras, entre otros, pueden afectar el resultado.

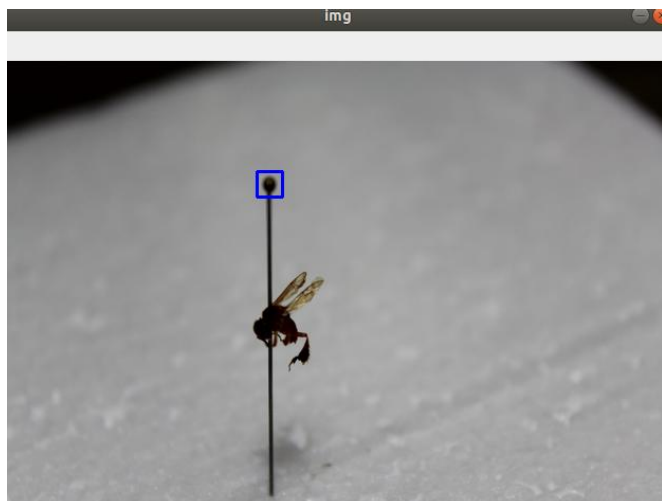


Figura 42 . Ejemplo Ruido 2

Fuente: Elaboración propia.

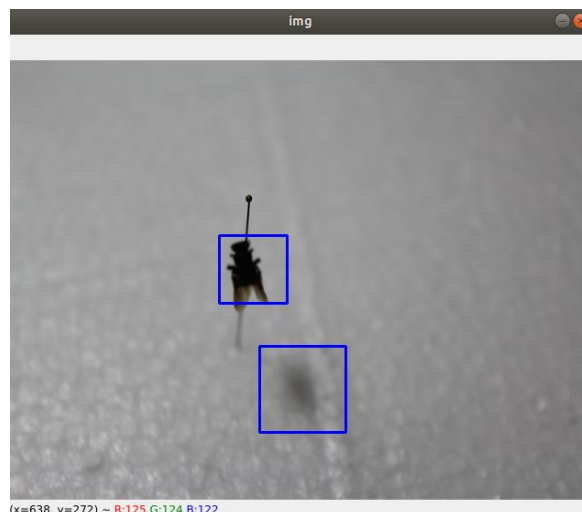


Figura 41 . Ejemplo Ruido 1

Fuente: Elaboración propia.

7. La aplicación implementada es una herramienta de apoyo, sin embargo requiere el acompañamiento, experiencia y observaciones de un taxónomo.
8. Con base en las entrevistas realizadas, el CINAT expone la utilidad de una herramienta que permita identificar, aunque no sea de manera específica en todos los casos, la especie que se analiza. El hecho de poder discriminar un porcentaje de entre las 59 especies posibles, lo consideran de gran valor ya que optimiza y reduce los tiempos de identificación de especies. Por ejemplo, al obtener 20 posibles especies de las 59 disponibles, lo que representa un 30% del total.

Limitaciones

1. Inicialmente se tuvo un banco de imágenes escaso al inicio, por lo que para solventarlo se recurrió al uso de imágenes de abejas de diferentes sitios web. Cabe destacar que se utilizaron única y exclusivamente para fines académicos. Debido a

la necesidad de contar con más imágenes, se acudió al CINAT para fotografiar las distintas especies de abejas.

2. El CINAT actualmente digitaliza imágenes de alta calidad tomadas por medio de un estereomicroscopio, que en comparación a las imágenes tomadas por un dispositivo móvil, a pesar de contar con buenas características técnicas, no tienen la suficiente claridad y acercamiento para poder vislumbrar características microscópicas, que solo son visibles mediante el uso del estereomicroscopio. A pesar de la limitación y debido al alcance del proyecto, se limitó al uso de imágenes tomadas con dispositivos móviles.
3. A pesar de obtener resultados aceptables, de acuerdo con la retroalimentación obtenida durante las entrevistas, existe un margen de error dentro de la aplicación de entre un 20% a un 40%, por lo que se recomienda siempre un acompañamiento, supervisión y experiencia de un taxónomo.

Recomendaciones

1. Ampliar y no limitar el uso de la aplicación solo a imágenes captadas por dispositivos móviles, ya que se podría analizar y recolectar resultados con imágenes de alta calidad como las que el CINAT actualmente digitaliza, a partir de enfocar 3 o más diferentes partes de la abeja y fotografiar cada una por separada, así finalmente juntar las tomas y formar una sola imagen detallada.
2. También se recomienda complementar la aplicación con insumos característicos no visibles por medio de imágenes como:
 - Región
 - Piquera
 - Comportamiento
 - Tipo características del polen

- Características de la miel
 - Comportamientos sociales
3. Esta aplicación se categoriza en una versión beta y se recomienda su uso con el acompañamiento de un taxónomo profesional para obtener resultados asertivos cuando se acude a una investigación.
 4. Se recomienda utilizar las imágenes digitalizadas por el CINAT debido a la precisión de los equipos. También se pueden realizar pruebas exhaustivas tomando en cuenta las siguientes variables:
 - Múltiples texturas de fondo.
 - Múltiples ángulos tomados por abeja.
 - Múltiples fondos de diferentes colores y o tonalidades.
 - Diferentes tipos de iluminación.

REFERENCIAS

REFERENCIAS

Apiary Book Ltd. (2018). Apiary Book. Recuperado de:
<http://www.apiarybook.com>

Árboles Mágicos. (2016): Fundación Árboles Mágicos. Recuperado de:
<http://www.arbolesmagicos.org>

Bagnato, J. (2018). ¿Cómo funcionan las Convolutional Neural Networks? Visión por ordenador. Recuperado de: <https://www.aprendemachinelearning.com/como-funcionan-las-convolutional-neural-networks-vision-por-ordenador/>

Bay, H., & Tuytelaars, T., & Van Gool L. (2006). SURF: Speeded Up Robust Features. Recuperado de: <https://www.vision.ee.ethz.ch/~surf/eccv06.pdf>

BBC. (2014). What is a tribe? Recuperado de: <http://www.bbc.co.uk/nature/tribe>

Botanical Online. (1999-2018). Propiedades de la miel. Recuperado de:
<https://www.botanical-online.com/mielpropiedades.html>

Calonder, M., & Lepetit, V., & Strecha, C. & Fua, P. (2010). “BRIEF: Binary Robust Independent Elementary Features”.

Camargo, J. & Pedro, S. (2007). Meliponini Lepeletier. En J. Moure y D. Urban (Eds.), Catalogue of bees (Hymenoptera, Apoidea) in the neotropical region (pp. 272-578). Curitiba: Sociedade Brasileira de Entomologia.

Capital México. (2017). Miel, oro líquido. Recuperado de:
<http://www.capitalmexico.com.mx/especial/miel-oro-liquido-produccion-belleza-salud-apicultura/>

Castillo, J. (2016) Ventajas de tener una app para tu negocio. Recuperado de:
<https://www.occ.com.mx/blog/tener-una-app-para-tu-negocio/>

CBD. (2007). Secretariat of Convention on Biological Diversity. (2007). Guide to the Global Taxonomy Initiative, CBD Technical Series.

Chavarría, M. (2015). Base de datos y georreferenciación de las colecciones de abejas sin aguijón de Costa Rica del Centro de Investigaciones Apícolas Tropicales de la Universidad Nacional de Costa Rica. 3.

Cordero, C. (2018). Sigue aumento de uso de Internet móvil en Costa Rica, pero cae la velocidad promedio de 4G. Recuperado de:
<https://www.elfinancierocr.com/tecnologia/sigue-aumento-de-uso-de-internet-movil-en-costa/6CDA3KNM6ND3BOC4BARGYKSGV4/story/>

Davison, M. (2016). Difference of Gaussians Edge Enhancement. Recuperado de
<http://micro.magnet.fsu.edu/primer/java/digitalimaging/processing/diffgaussians/index.html>

De Oliveira Faria, A. (2018). Deep Learning in openCV for visually impaired. Recuperado de: <https://github.com/cabelo/youreyes>

Django (2021). Meet Django. Recuperado el 30 de marzo del 2021 de: <https://www.djangoproject.com/>

Ecos del Bosque (2021). Recuperado el 31 de marzo del 2021 de: <https://ecosdelbosque.com/>

Espinoza, F., & Padilla, S., & Hernández, P., & Benítez, J., & Zamora, L. (2015). Guía práctica de identificación de abejas nativas sin aguijón (Apidae, Meliponini) por medio de sus entradas. Universidad Nacional de Costa Rica.

Fielding, R. (1999). RFC2616. Recuperado de: <https://tools.ietf.org/html/rfc2616>

Figueroa G., & Prendas J., & Ramírez M., Aguilar I., & Herrera E., & Travieso C. (2015) Identificación de abejas sin aguijón (Apidae: Meliponini) a partir de la clasificación de los descriptores SIFT de una imagen del ala derecha anterior.

Git (2021). Recuperado el 30 de marzo de 2021 de: <https://git-scm.com/>

Gorini, M. (2019). ¿Cuál es la diferencia entre el machine larning y el Deep learning? Recuperado de: <https://blog.bismart.com/es/diferencia-machine-learning-deep-learning>

Guerra, J. (2014). Bases de Datos en Taxonomía y Colecciones Científicas. Recuperado de: <https://es.slideshare.net/arturogcantu/bases-de-datos-en-taxonoma-y-colecciones-cientificas>

Hernández, S. (2015). Aplicaciones Médicas Móviles: definiciones, beneficios y riesgos. Recuperado de:
<http://rcientificas.uninorte.edu.co/index.php/salud/article/viewArticle/7622/8567>

Ionic (2021). Recuperado el 30 de marzo del 2021 de: <https://ionicframework.com/>

IoT for All (2017). Computer Vision Technology Permeates Our Daily Lives. Recuperado de: <https://www.iotforall.com/computer-vision-applications-in-daily-life/>

Lapedriza, Á. (2012). Universidad Oberta de Catalunya, La visión por computadora, una disciplina en auge. Recuperado de:
<http://informatica.blogs.uoc.edu/2012/04/19/la-vision-por-computador-una-disciplina-en-auge/>

Lowe, D. (2004). Distinctive Image Features from Scale-Invariant Keypoints. Recuperado de <https://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf>.

Mansilla, N. (2015). Procesamiento de imágenes aplicada a la tipificación vacuna: análisis de indicadores geométricos basados en curvatura.

Martínez, J., & Merlo, F. (2014) Importancia de la diversidad de abejas (Hymenoptera: Apoidea) y amenazas que enfrenta en el ecosistema tropical de Yucatán, México. Recuperado de: http://www.scielo.org.bo/pdf/jsaas/v1n2/v1n2_a03.pdf.

Matheson R. (2018). Fleets of drones could aid searches for lost hikers. Recuperado de: <http://news.mit.edu/2018/fleets-drones-help-searches-lost-hikers-1102>.

Meliponicultura (2014). Manejo de Colmenas Meliponas Silvestres en Costa Rica. Recuperado de:
https://issuu.com/abejassilvestres2013/docs/escrito_manejo_colmenas_silvestres_

Michener, C. (2007). The bees of the world. New York: Johns Hopkins University Press.

Miel Dorada (2017). Recuperado el 31 de marzo del 2021 de:
<https://www.mieldorada.net>

Mordvintsev, A. (2013). ORB (Oriented FAST and Rotated BRIEF). Recuperado de https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_feature2d/py_orb/py_orb.html

OpenCV (2021). Recuperado el 31 de marzo de 2021 de: <https://opencv.org/>

Ortega, D., & Guevara, M., & Benavides, J. (2017). Elementary: Un Framework de programación web. Recuperado de: <https://www.redalyc.org/pdf/784/78457627004.pdf>

Pacheco, Álvarez (2018). Sin abejas no hay agricultura: Costa Rica debe prohibir los neonicotinoides. Recuperado de: <https://semanariouniversidad.com/opinion/sin-abejas-no-hay-agricultura-costa-rica-debe-prohibir-los-neonicotinoides/>

Peña, M. (2018). ¿Para qué usa usted su teléfono celular? Recuperado de:
<https://www.ucr.ac.cr/noticias/2018/01/31/para-que-usa-usted-su-telefono-celular.html>

Prendas, P. (2015). <https://repositoriotec.tec.ac.cr/handle/2238/6448>. Recuperado de: <https://repositoriotec.tec.ac.cr/handle/2238/6448>

Python (2021). What is Python? Executive Summary. Recuperado el 30 de marzo del 2021 de: <https://www.python.org/doc/essays/blurb/>

Rosten, E., & Porter, R., & Drummond, T. (2008). Faster and better a machine learning approach to corner detection.

Roussell, J. (2018). China can apparently now identify citizens based on the way they walk. Recuperado de: <https://techcrunch.com/2018/11/07/china-can-apparently-now-identify-citizens-based-on-the-way-they-walk/>

Santiago, R., & Trbaldo, S., & Fernández Á. (2015). Mobile learning, nuevas realidades en el aula.

Slaa, J., & Sánchez, L., & Braga, K., & Hofstede, Frouke. (2006). Stingless bees in applied pollination: Practice and perspectives. Recuperado de:

https://www.researchgate.net/publication/41713812_Stingless_bees_in_applied_pollination_Practice_and_perspectives

Szeliski R. (2010). Computer Vision: Algorithms and Applications.

Tabor, N. (2018). Smile! The Secretive Business of Facial-Recognition Software in Retail Stores. Recuperado de: <http://nymag.com/intelligencer/2018/10/retailers-are-using-facial-recognition-technology-too.html>

Tehreem, N. (2021). REST API Definition: What is a REST API (RESTful API)?
Recuperado de: <https://www.astera.com/type/blog/rest-api-definition/>

Vaca, C. (2011). Paradigmas de programación. Recuperado de:
<https://www.infor.uva.es/~cvaca/asigs/docpar/intro.pdf>

Valdivia, A. (2016). Diseño de un Sistema de Visión Artificial para la
Clasificación de Chirimoyas basado en medidas.

Vale, J., & Gutierrez, G., & J. Gildardo. (2005). Definición arquitectura cliente
servidor. Recuperado de:
https://www.ecotec.edu.ec/documentacion/investigaciones/docentes_y_directivos/articulos/5743_TRECALDE_00212.pdf

Villalobos, J. (2018). Universidad Nacional, Imaging Lab. Recuperado de:
<http://www.imaginglab.una.ac.cr>

Anexo 1 – Manual de Usuario

Manual de usuario

Versión 1.0

Fecha: 1 de abril de 2021

Notas para el administrador y/o desarrollador

- Esta es una versión beta de la aplicación.
- Se recomienda su uso en acompañamiento con un taxónomo experto.
- Se recomienda su uso a discreción del CINAT y de la Escuela de Informática de la Universidad nacional.
- No se incluye el proceso de instalación de las tecnologías a nivel de backend y frontend, las cuales son Django y Ionic respectivamente, sin embargo se adjuntan las referencias hacia su documentación actual en la sección de “Referencias Útiles”, las cuales se recomiendan altamente ser consultadas para el debido despliegue del sistema, tanto en un ambiente de desarrollo y pruebas, como de producción.
- Con base en el código fuente proveído, se puede realizar la generación de la aplicación tanto para dispositivos móviles en plataformas Android y iOS, como también a nivel de aplicación web. Se recomienda consultar la documentación oficial de Ionic para los pasos requeridos. Consultar la sección de “Referencias Útiles” de este documento.
- También se brinda la última versión de los requerimientos instalados para el backend, en cuanto a librerías a nivel de ambiente virtual de desarrollo de Python:
 - django==3.0.6
 - libmysqlclient-dev
 - mysqlclient
 - mysql-server
 - djangorestframework
 - markdown
 - django-filter
 - Output from "pip freeze" command
 - asn1crypto==0.24.0
 - cryptography==2.1.4
 - enum34==1.1.6
 - gyp==0.1
 - idna==2.6
 - ipaddress==1.0.17
 - keyring==10.6.0
 - keyrings.alt==3.0
 - mysql-connector-python==2.1.6
 - mysql-utilities==1.6.4
 - paramiko==2.0.0
 - pexpect==4.2.1
 - pyasn1==0.4.2
 - pycrypto==2.6.1

- pygobject==3.26.1
 - pyodbc==4.0.17
 - pysqlite==2.7.0
 - pyxdg==0.25
 - SecretStorage==2.3.1
 - six==1.11.0
- En cuanto al entrenamiento del algoritmo Cascade Classifier, se proveen recursos que explican cómo realizarlo y adicionalmente referencias hacia su documentación oficial. Consultar la sección de “Referencias Útiles” de este documento.
- Se incentiva a brindar la continuidad deseada o requerida del sistema, con tal de brindar una herramienta con muchas más características, funcionalidades y seguridad deseada. Así como tomar en cuenta las conclusiones, recomendaciones y limitaciones expuestas en la tesis que se ostenta con esta aplicación.
- En cuanto a conocimientos técnicos requeridos para darle continuidad a la aplicación y/o soporte, se recomiendan:
 - Conocimiento en Python, Angular, HTML5, SASS, SQL.
 - Conocimiento en REST y JSON.
 - Conocimiento en Django y Ionic.
 - Conocimiento básico en redes y servidores, con la intención de realizar el despliegue del sistema en un ambiente completo y seguro de producción.
- Se detalla a continuación la guía de cómo realizar las diferentes tareas tanto de mantenimiento del sistema como de aprovechamiento en cuanto a las demás características disponibles.

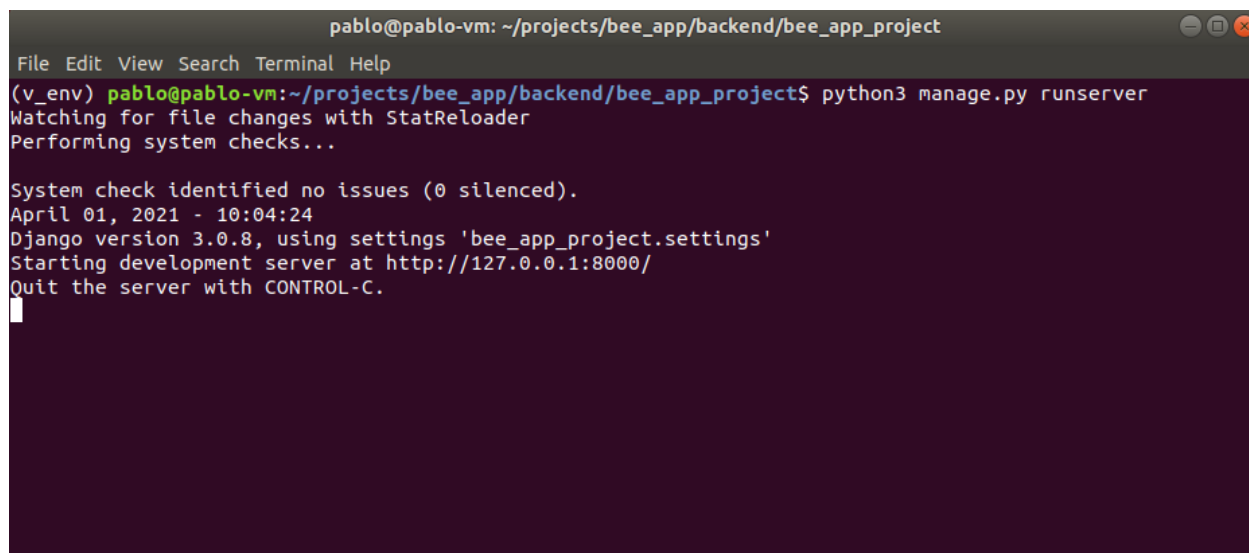
Inicio

Para dar inicio al sistema como un todo, se debe iniciar primeramente el servidor con Django instalado. Para ello a manera de ejecución de levantamiento de Django, se debe abrir una terminal y acceder al directorio “bee_app/backend/bee_app_project”.

Una vez ahí, ejecutar “python3 manage.py runserver”.

NOTA, se recomienda, en caso de ejecución dentro de un ambiente de desarrollo, a utilizar virtual environments de Python. Además, se puede consultar en el directorio “bee_app/backend/bee_app_project/requirements.txt” los últimos paquetes instalados para la debida ejecución del sistema sin problemas.

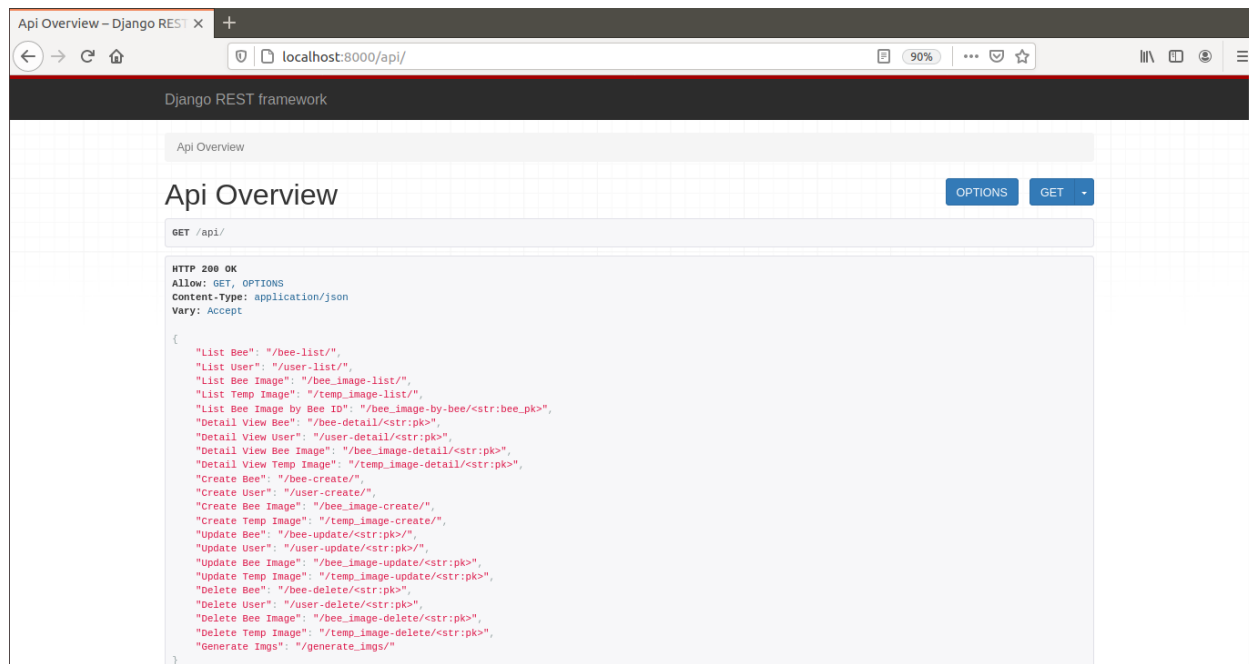
De esta manera se debería ver un resultado como el siguiente:



```
pablo@pablo-vm: ~/projects/bee_app/backend/bee_app_project
File Edit View Search Terminal Help
(v_env) pablo@pablo-vm:~/projects/bee_app/backend/bee_app_project$ python3 manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
April 01, 2021 - 10:04:24
Django version 3.0.8, using settings 'bee_app_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

Para realizar una breve verificación, se puede consultar el índice del RESTful API creado, accediendo a <http://localhost:8000/api> a través del browser de preferencia. Se obtendría una vista como la siguiente:



Una vez verificado e iniciado Django, se procede a iniciar Ionic. Para ello, abrimos otra terminal y nos movemos al directorio “bee_app/frontend/bee_app”, y ejecutamos “Ionic serve”, obteniendo una vista como la siguiente:

```
pablo@pablo-vm: ~/projects/bee_app/frontend/bee_app
File Edit View Search Terminal Tabs Help
pablo@pablo-vm: ~/projects/bee_app/backend/bee_app... x pablo@pablo-vm: ~/projects/bee_app/frontend/bee_app x
module.js, update-user-details-update-user-details-module.js.map (update-user-details-update-user-detail
s-module) 13.7 kB [rendered]
[ng] chunk {update-user-update-user-module} update-user-update-user-module.js, update-user-update-user
-module.js.map (update-user-update-user-module) 10.1 kB [rendered]
[ng] chunk {users-users-module} users-users-module.js, users-users-module.js.map (users-users-module)
10.9 kB [rendered]
[ng] chunk {vendor} vendor.js, vendor.js.map (vendor) 5.44 MB [initial] [rendered]
[ng] Date: 2021-04-01T10:07:53.993Z - Hash: bc75573b45307b8f653b - Time: 82265ms
[ng] WARNING in /home/pablo/projects/bee_app/frontend/bee_app/src/test.ts is part of the TypeScript co
mpilation but it's unused.
[ng] Add only entry points to the 'files' or 'include' properties in your tsconfig.
[ng] WARNING in /home/pablo/projects/bee_app/frontend/bee_app/src/app/bees/bees.model.ts is part of th
e TypeScript compilation but it's unused.
[ng] Add only entry points to the 'files' or 'include' properties in your tsconfig.
[ng] WARNING in /home/pablo/projects/bee_app/frontend/bee_app/src/environments/environment.prod.ts is
part of the TypeScript compilation but it's unused.
[ng] Add only entry points to the 'files' or 'include' properties in your tsconfig.

[INFO] Development server running!

Local: http://localhost:8100

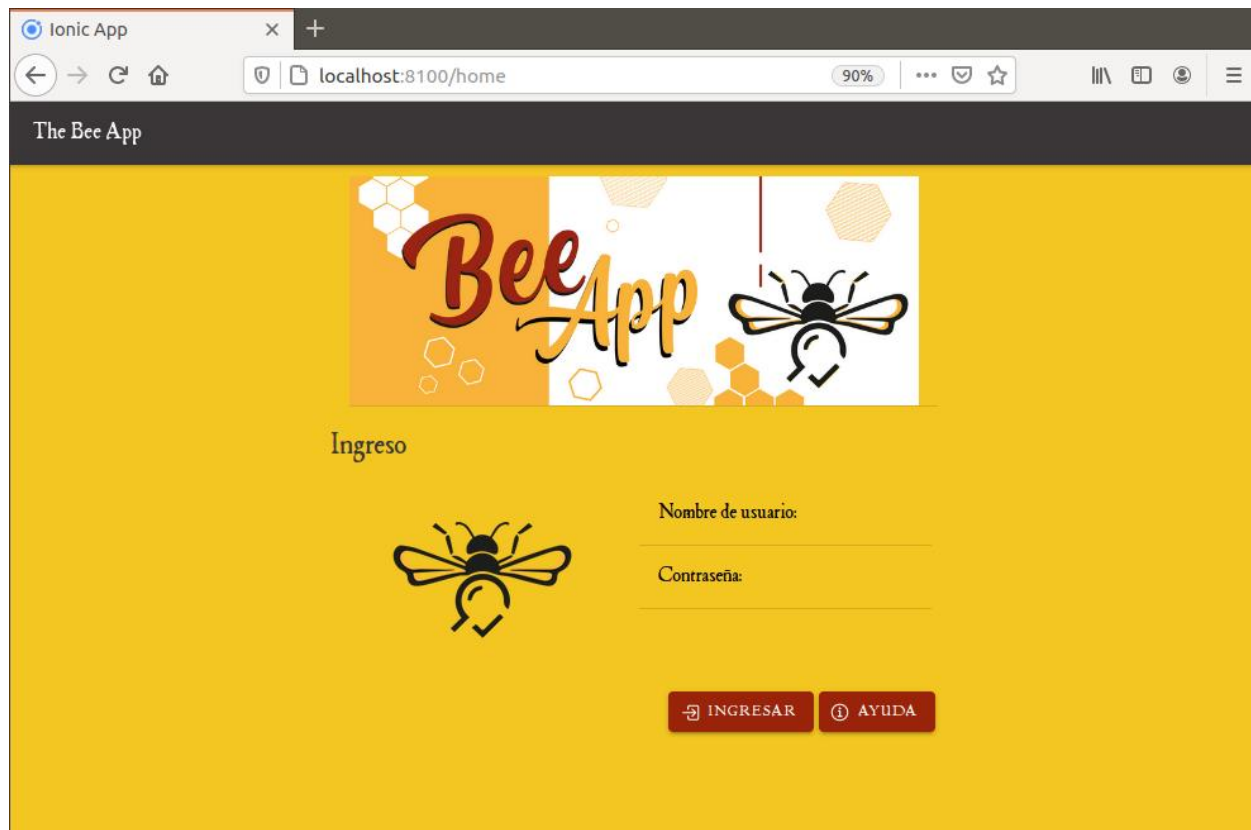
Use Ctrl+C to quit this process

[INFO] ... and 80 additional chunks
[ng] : Compiled successfully.
[INFO] Browser window opened to http://localhost:8100!
```

Si se accede a la dirección <http://localhost:8100> se debería ser re direccionado al home de la aplicación.

Login

Como parte del acceso a la aplicación, una vez se ingresa al home, se obtiene una vista de acceso como la siguiente:



Una vez acá, basta con ingresar los credenciales pertinentes y dar click en INGRESAR.

Logout

Para salir de la aplicación, una vez se está “logueado”, basta con dar click en la esquina superior derecha:



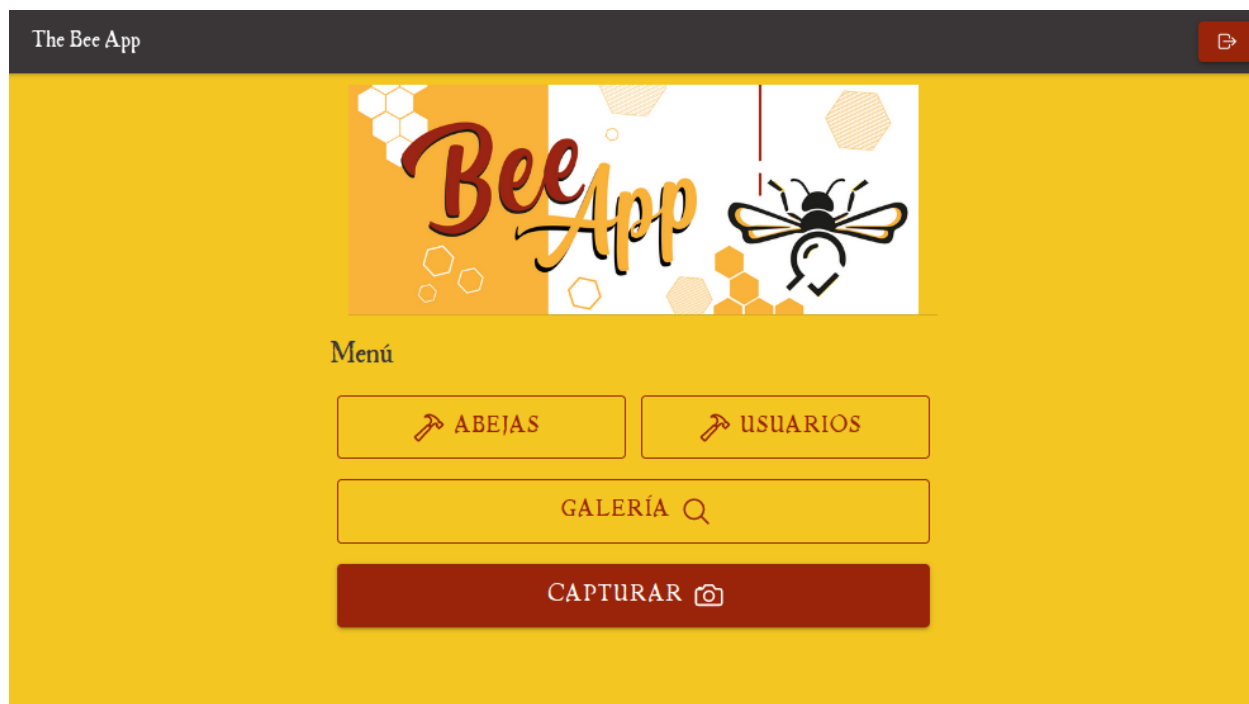
Ayuda

En caso de requerir ayuda para el acceso de la aplicación, se puede dar click al botón de ayuda en la pantalla inicial del home, la cual contiene información referente para contactar al administrador del sistema y/o CINAT.



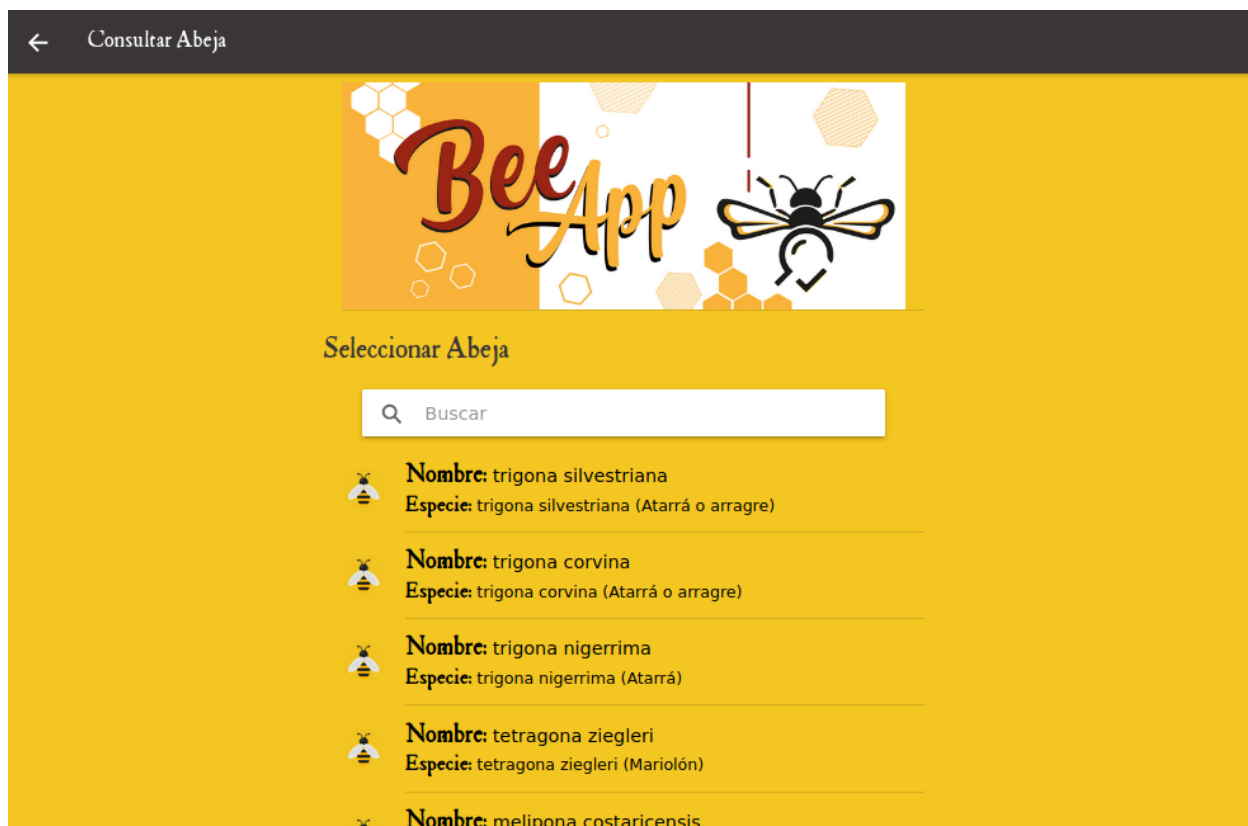
Menú Principal

Una vez “logueados” dentro de la aplicación, se pueden observar las diferentes opciones a utilizar:



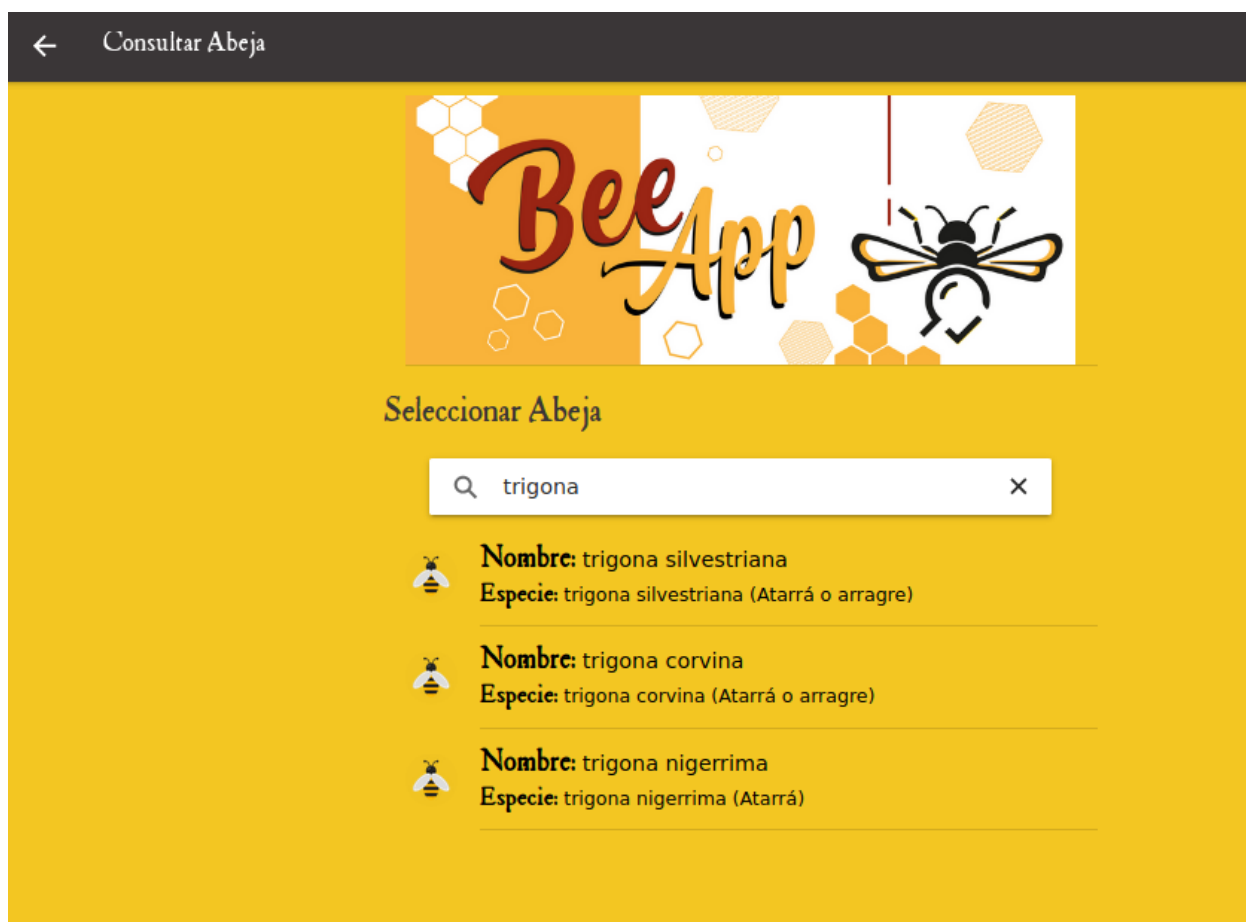
Consultar Galería

Si damos click en GALERÍA, se podrá acceder a la lista actual de abejas dentro de la base de datos del sistema:



Herramienta de filtrado de búsqueda

Actualmente, la aplicación permite, si se tienen muchas entradas a nivel de base de datos y con respecto a abejas, a utilizar el campo de búsqueda, el cual permite realizar un filtrado dentro de la lista de abejas por Especie o Nombre:



Una vez se identifique la abeja que se quiere consultar, se da click sobre ella:

← Detalle de la Abeja



trigona nigerrima

Especie: trigona nigerrima (Atarrá)
Región: Norte
Descripción:

Morfología observable: Abeja mediana; cabeza y tórax de color negro con vellosidad; abdomen y patas café oscuro. Posee alas negras. Presenta pelos muy grandes en el clípeo (en la parte

De esta manera se pudo observar y consultar la ficha técnica actual de la abeja. Permite también desplazar las imágenes, de manera que se pueda visualizar la colección actual de dichas imágenes de la abeja.

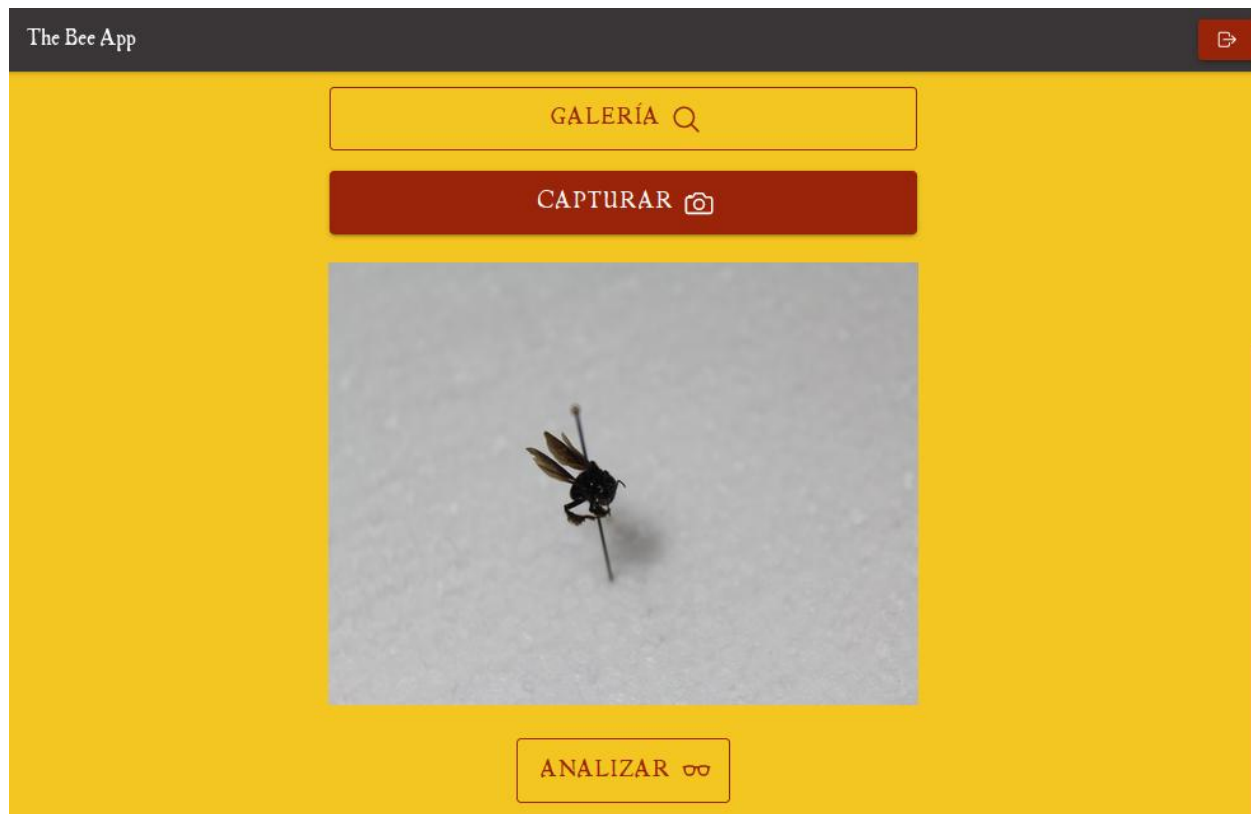
Para salir de esta opción y volver al menú principal, basta con dar click en la flecha blanca, en la parte superior izquierda hasta movernos al menú deseado.

Capturar y Analizar

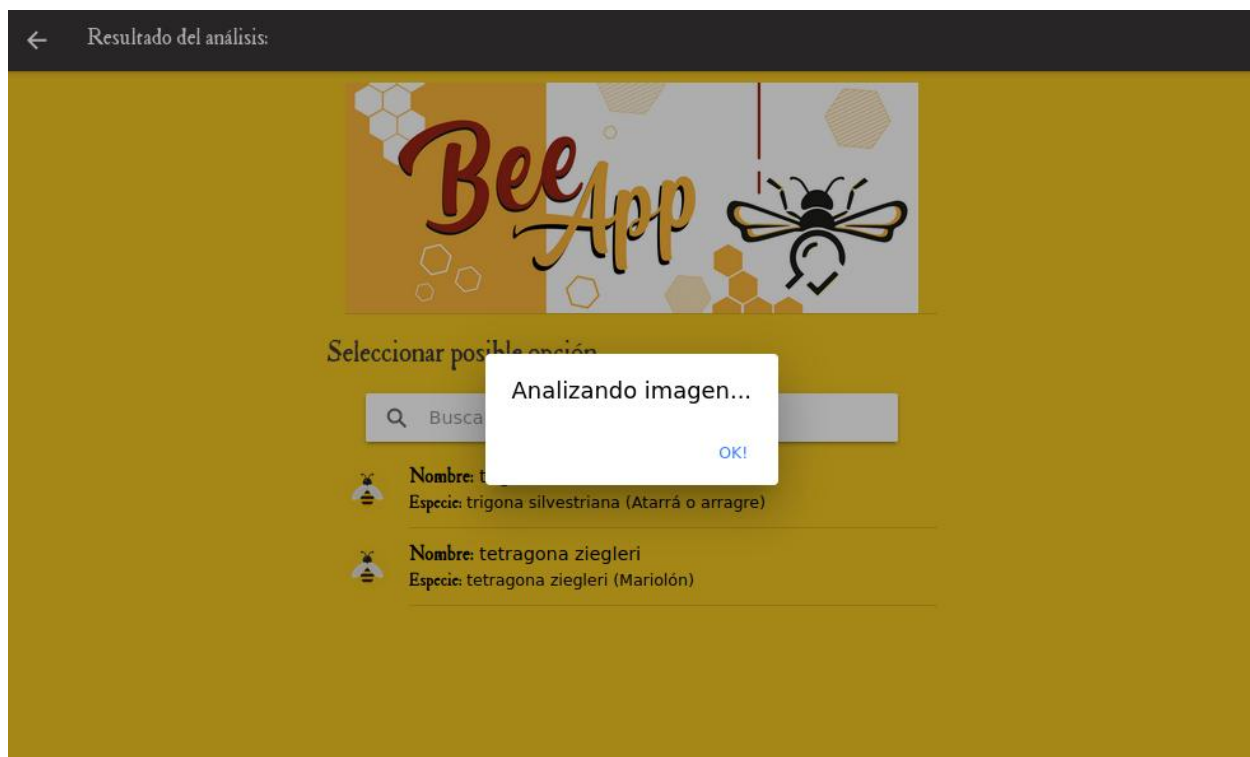
Para realizar una captura, se procede a seleccionar desde el menú principal la opción pertinente:



Esto nos va a habilitar 2 posibles opciones, si se cuenta con cámara a nivel de hardware, se abrirá dicha cámara y permitirá tomar la captación de la fotografía de la abeja. En caso contrario, sino se cuenta con cámara, se puede realizar la carga de una imagen previamente capturada y almacenada en el carrete.



Con la imagen cargada, procedemos a dar click en ANALIZAR, para enviar la imagen al servidor y obtener una respuesta con base a que especie de abeja puede ser:



Damos click en OK!

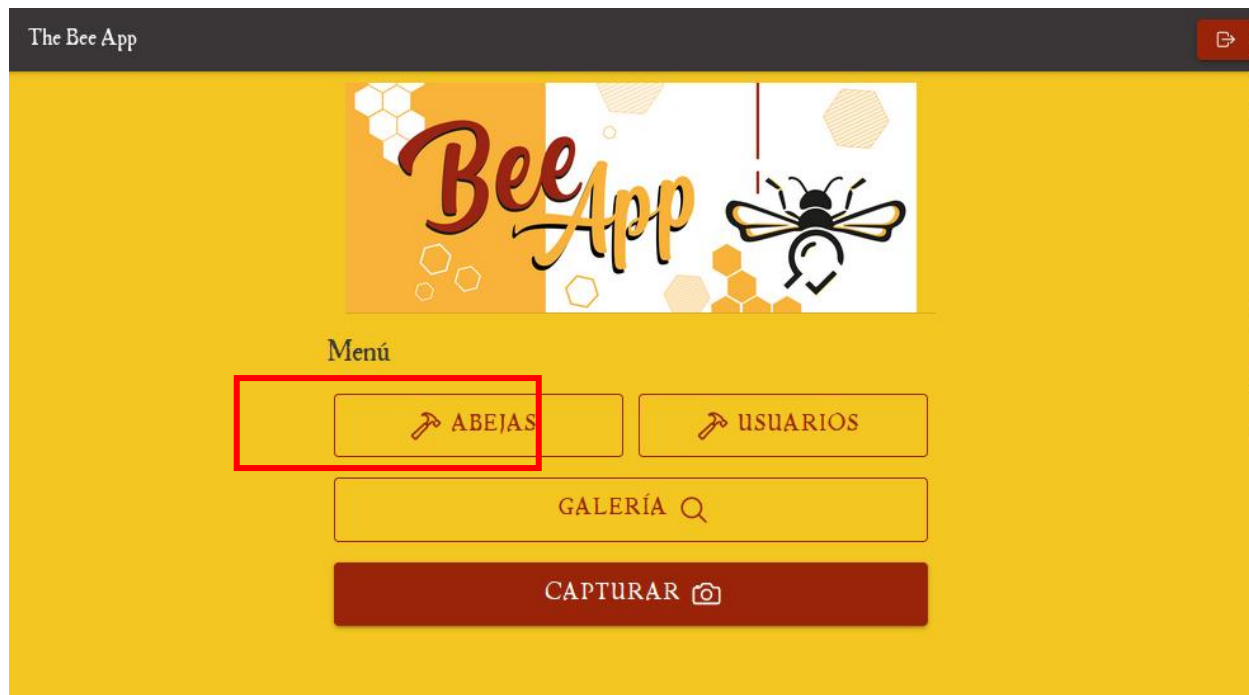
Y accedemos a la abeja:



- **NOTA:** Como se acotó en las notas de este documento inicialmente, se recomienda tomar en cuenta las conclusiones, recomendaciones y limitaciones expuestas en la tesis que se ostenta con esta aplicación.

Una vez se haya finalizado y revisado las posibles opciones ante las cuales podría encontrarse la abeja enviada para análisis, podemos volver a realizar otro análisis utilizando la flecha de regreso, en la esquina superior izquierda o volver al menú principal.

Nuevamente en el menú principal, para poder realizar los debidos mantenimientos del sistema, se procede a ingresar al Menú de Abejas, dando click en el botón correspondiente.



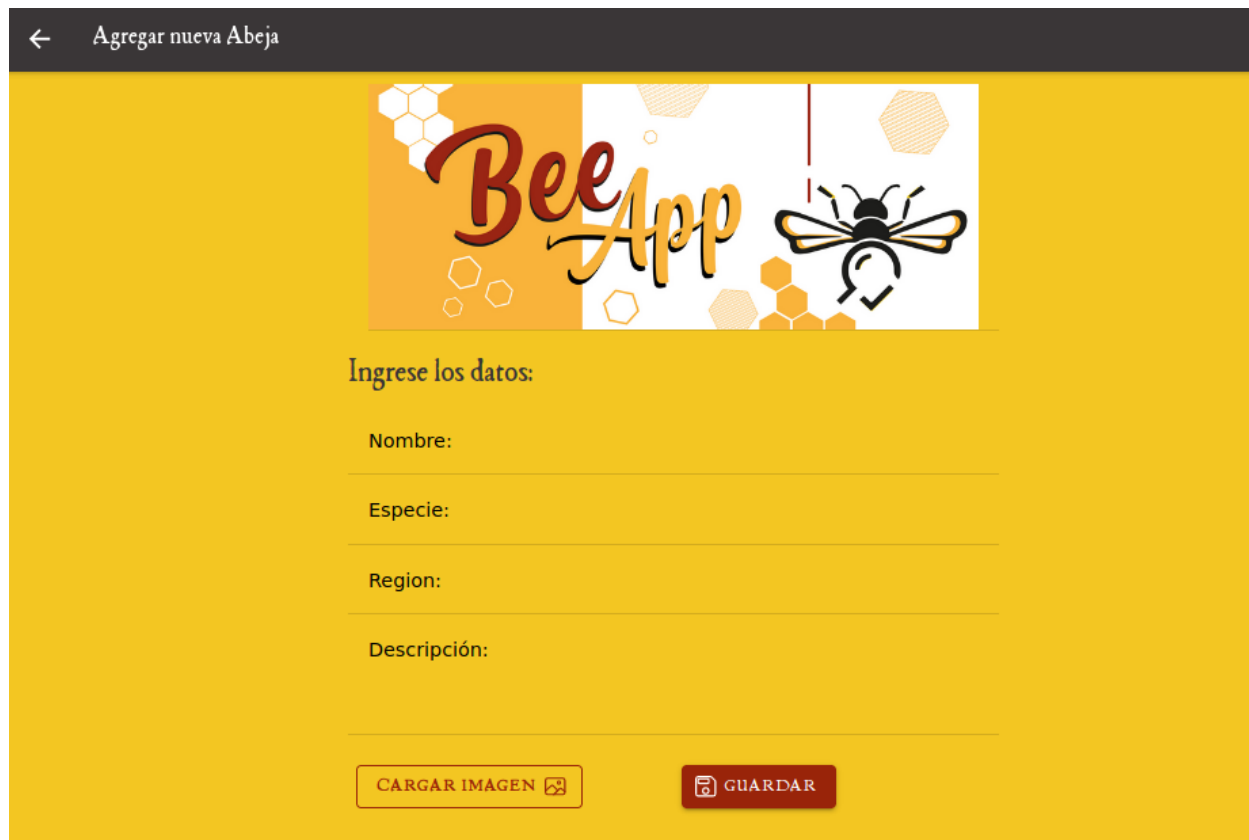
Menú de Abejas

Dentro de este menú se tienen las diferentes opciones para mantenibilidad y gestión del sistema:



Agregar Abeja

Si se desea agregar una abeja, se da click sobre la opción AGREGAR ABEJA y posteriormente, se procede a llenar la información relacionada a dicha abeja:



The screenshot shows a mobile application interface for adding a new bee. At the top, there is a dark grey header bar with a back arrow and the text 'Agregar nueva Abeja'. Below the header is a large yellow banner featuring the 'Bee App' logo in a stylized font, surrounded by hexagonal patterns and a bee illustration. Underneath the banner, the text 'Ingrese los datos:' is displayed. Below this, there are four input fields with labels: 'Nombre:', 'Especie:', 'Region:', and 'Descripción:'. At the bottom of the form, there are two buttons: 'CARGAR IMAGEN' with a camera icon and 'GUARDAR' with a save icon.

Se permite realizar la carga inicial de una imagen asociada, posteriormente, mediante la opción ACTUALIAR ABEJA, se pueden agregar todas las imágenes demás deseadas.

Una vez el ingreso de la información finaliza, se le da click a la opción GUARDAR.

Consultar Abeja

Posteriormente a la creación de una abeja, podemos dar click a la opción CONSULTAR ABEJAR e ingresar a la lista de abejas actual dentro de la base de datos. Se puede hacer uso del filtro de búsqueda anteriormente mencionado y dar click en la abeja deseada para ser consultada. Una vez finalizada la consulta, se regresa al Menú de Abejas o el Menú principal.

Actualizar Abeja

Para actualizar una abeja, se selecciona la opción correspondiente y dentro de la lista resultante, se selecciona la abeja que se desea actualizar:

←

Actualizar Abeja



+

×

Ingrese los nuevos datos:

Nombre:

melipona costaricensis

Especie:

melipona costaricensis (Jicote barcino)

Region:

Limón, Puntarenas, Guanacaste

Descripción:

Morfología observable:

Una vez acá adentro, se pueden modificar los datos a gusto y también agregar o eliminar imágenes asociadas a dicha abeja, mediante los botones proveídos debajo del carrusel de imágenes.

Una vez finalizada la actualización, se procede a dar click al botón ACTUALIZAR:

 Actualizar Abeja







Ingrese los nuevos datos:

Nombre:

melipona costaricensis

Especie:

melipona costaricensis (Jicote barcino)

Region:

Limón, Puntarenas, Guanacaste

Descripción:

Morfología observable:

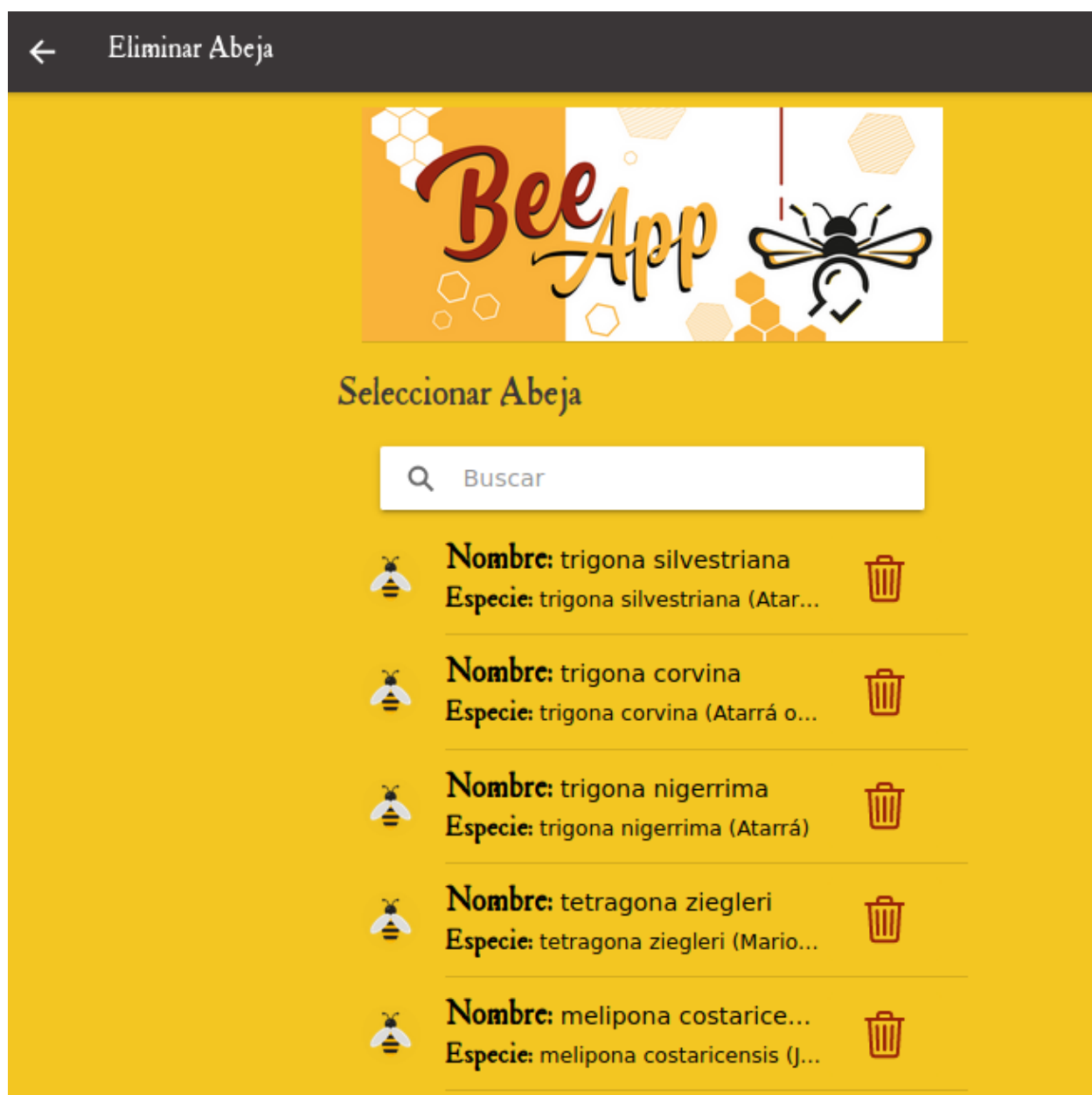
Abejas ligeramente más pequeñas que M

 ACTUALIZAR

De esta manera regresaríamos al listado de abejas y se puede continuar actualizando más abejas o regresar al Menú de abejas.

Eliminar Abeja

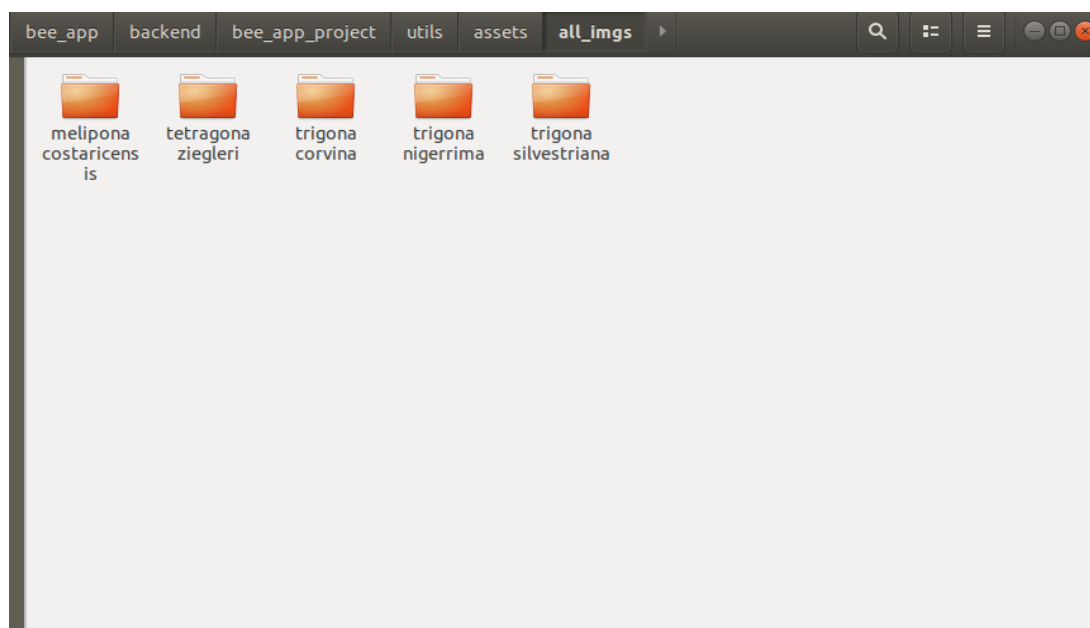
Dentro del Menú de abejas, se selecciona la opción correspondiente para proceder a eliminar una abeja dentro del sistema.



Dentro de esta opción, basta con dar click en el ícono a la derecha del nombre de la abeja, para así desechar una abeja actual del sistema.

Generar Imágenes

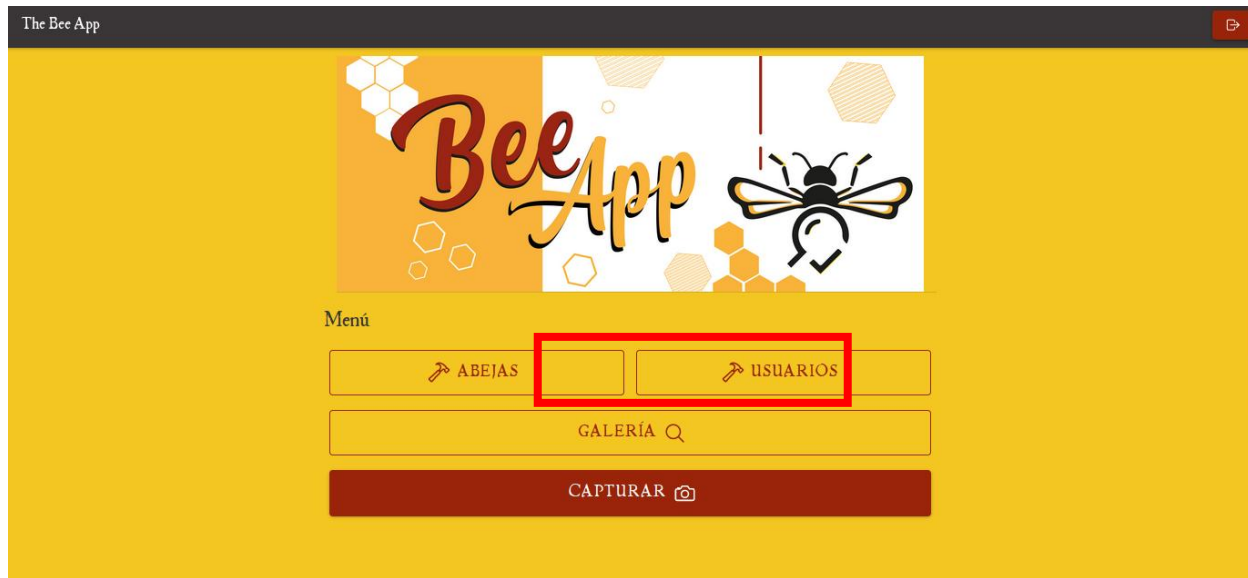
Esta opción dentro del Menú de Abejas, permite tomar todas las imágenes actuales que posee la base de datos con respecto a las abejas y, ya que están en un formato base64, convertirlas todas a png, ordenadas por directorios referentes a cada especie:



Dichas imágenes son generadas del lado del servidor en el directorio “bee_app/backend/utils/assets/all_imgs”. Una vez generadas pueden ser utilizadas como se requiera.

Menú de Usuarios

Para acceder al Menú de Usuario, se accede primeramente al Menú Principal de la aplicación y se le da click al botón correspondiente:



Tipos de Usuarios

Cabe mencionar que actualmente el sistema fue creado con dos tipos de usuarios, administrador y regular.

El usuario administrador tiene acceso a todas las opciones de los diferentes menús del sistema, mientras que el regular, posee una vista más sencilla y limitada, con el fin de que solo pueda consultar la galería y analizar capturas:



Agregar Usuarios

Para agregar Usuarios, se debe contar con permiso de administrador, posteriormente al haber ingresado al Menú de Usuarios, se selecciona la opción AGREGAR USUARIO y luego se llenan los datos referentes para el usuario a crear:

← Agregar nuevo Usuario



Ingrese los datos:

Nombre de usuario:

Contraseña:

Tipo:

GUARDAR

Una vez finalizado, se da click en GUARDAR.

Consultar Usuarios

Para consultar usuarios, desde el Menú de usuario, se selecciona la opción,
CONSULTAR USUARIO:

← The Bee App



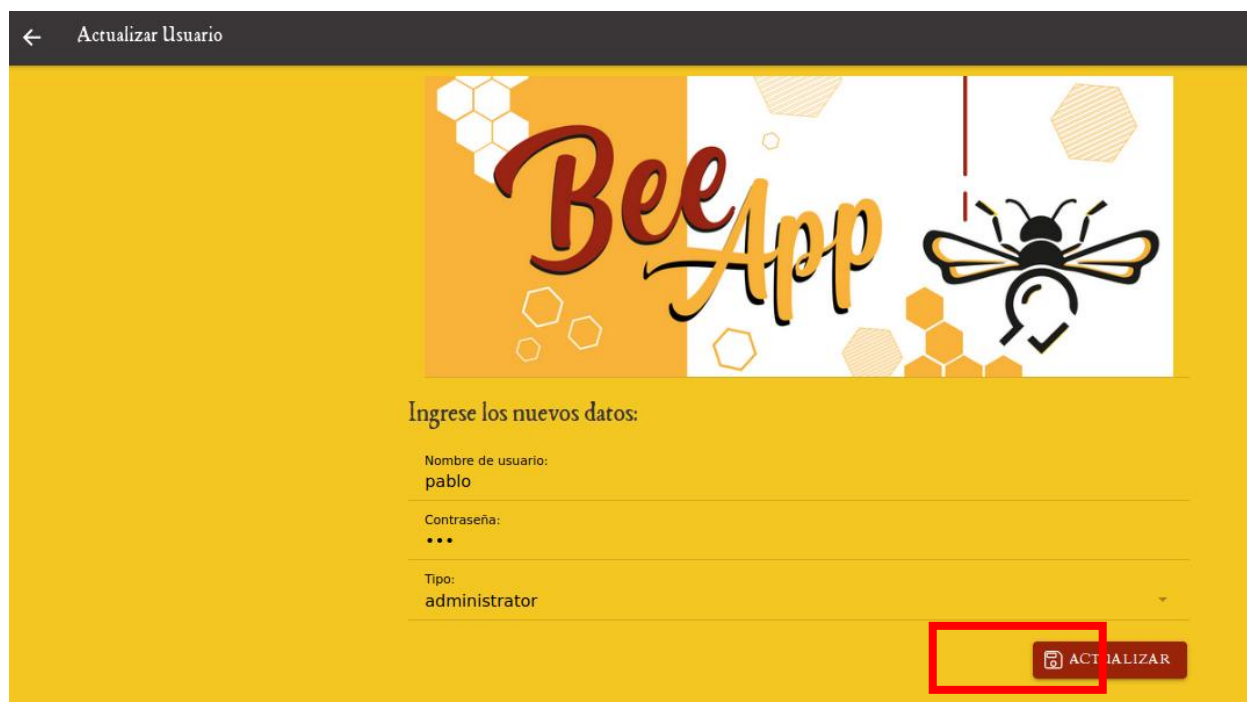
Menú Usuarios (Mantenimiento)

+ AGREGAR USUARIO	Q CONSULTAR USUARIO
✓ ACTUALIZAR USUARIO	ELIMINAR USUARIO

Una vez se selecciona dicha opción, se procede a dar click sobre el usuario deseado a consultar.

Actualizar Usuario

Para actualizar un usuario, desde el menú principal se procede a la opción ACTUALIZAR USUARIO, y desde la nueva vista, se actualizan cada uno de los antiguos valores deseados. Una vez finalizado, se procede a dar click en ACTUALIZAR:



← Actualizar Usuario

Bee App

Ingrese los nuevos datos:

Nombre de usuario:
pablo

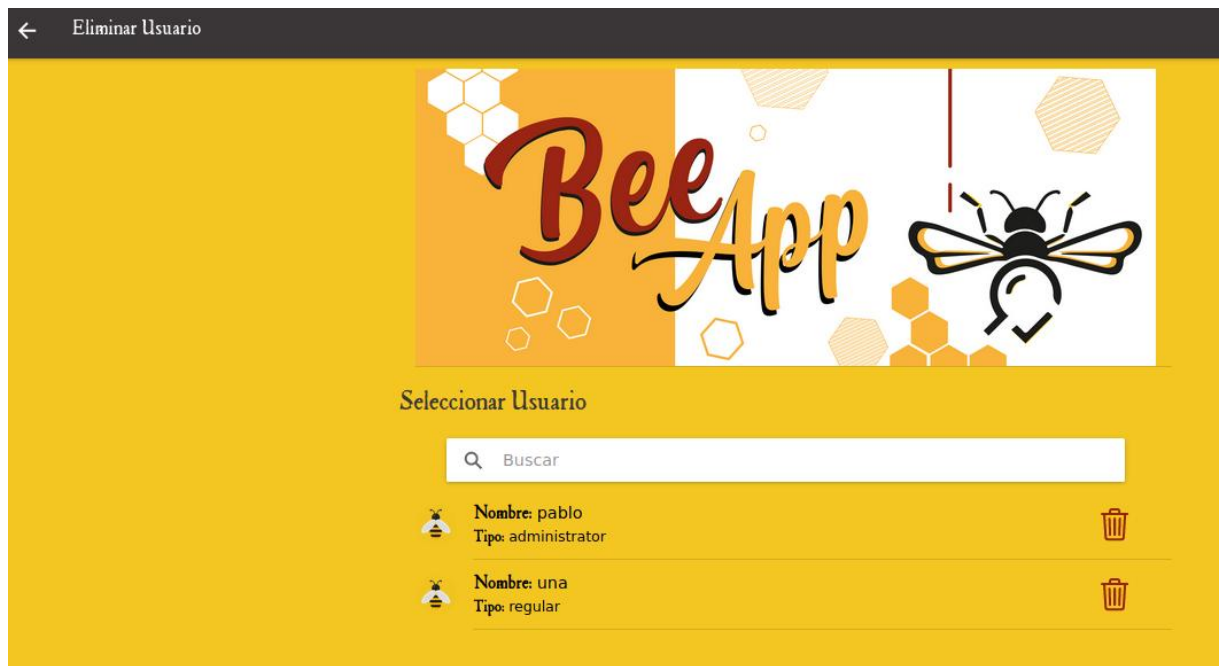
Contraseña:
...

Tipo:
administrator

ACTUALIZAR

Eliminar Usuarios

Para eliminar un usuario, desde el menú de usuarios, se procede a seleccionar la opción de ELIMINAR USUARIO. Posteriormente dentro de la lista, se da click en el ícono a la derecha del nombre con la finalidad de escoger el usuario a ser eliminado:



Referencias útiles

API Resfull completo en Django	https://www.youtube.com/channel/UCZ4IdHv04_c7mjW5Q9FWrgQ/videos
Cómo desplegar django	https://www.a2hosting.com/kb/developer-corner/python/installing-and-configuring-django-on-linux-shared-hosting
Cómo desplegar Ionic app móvil	https://ionicframework.com/docs/v3/intro/deploying/
Cómo desplegar ionic web	https://ionicframework.com/docs/deployment/progressive-web-app
Convertir base64 a png	https://moonbooks.org/Articles/How-to-convert-a-base64-image-in-png-format-using-python-/
Entrenamiento y aplicación de haar cascade	https://www.youtube.com/watch?v=POSYDLcspIk

Haar Cascade	https://docs.opencv.org/3.4/db/d28/tutorial_cascade_classifier.html
Open CV, ejemplo básico	https://www.youtube.com/watch?v=LopYA64KmdE
Sitio y documentación oficial de Django	https://www.djangoproject.com/
Sitio y documentación oficial de Ionic	https://ionicframework.com/
Sitio y documentación oficial de OpenCV	https://opencv.org/
Utilidades Ionic css	https://ionicframework.com/docs/layout/css-utilities